

Project Report on

“Markov Chain Analysis of Indian Classical Music and Tool for the Prediction of the Raaga of the Songs”

Submitted by

Amit Mondal

Ankush Karmarkar

Atharva Thatte

Prasanna Dillikar

Pushkar Kale

Samarth Dandawate

Under the Guidance of

Prof. S.S. Shende



Fergusson College, Pune

Certificate

This is to certify that Mr.
(Roll No. _____) B.Sc. final year student of Department
of Statistics, Fergusson College, Pune has successfully
completed the project as a part of curriculum in 2017-
2018.

Prof. S. S. Shende
Project Guide

Prof. S. S. Shende
Head, Dept. of Statistics

Acknowledgement

We have taken a lot of efforts in this project. However, it would not have been possible without the support and help of many individuals. We would like to extend a big thanks to all of them. We are highly indebted to Prof. Subhash Shende, HOD, Dept. of Statistics, Fergusson college, Pune for his guidance and consistent supervision as well as for teaching the concepts regarding the project and encouraging us for this work. We would also like to express gratitude to all the Teachers of Department of Statistics, Fergusson College for their support and kind co-operation. Also, we are thankful to the Non-teaching staff of the Department.

The person who deserved most appreciation is Atharva Vaidya, an Engineering student from Sinhgad College of Engineering, Pune, who developed a program based on our algorithm. Without his help, it would have been really difficult to complete our work in time. We are really overwhelmed with his efforts and support. We would also like to thank Mrunmayi Naik, our classmate of T.Y.B.Sc. Class, for providing us the material about the Indian Classical Music.

Our appreciations also go to our all friends who have willingly helped us out with their abilities. We are eternally grateful to our parents and family for their strong backing and promising support. Words are not enough to describe our gratitude to them.

Abstract

The two foundation elements of Indian classical music are *Raag* and *Taal*. Each *raag* is a unique sequence of melodic notes. As these *raagas* have a specific set of rules, they can be analysed mathematically.

The purpose of this study was to develop a tool to identify the *raag* of any song using the theory of stochastic processes. A total of 8 popular *raagas* were selected for the study.

A sample of two to three songs of each *raag*, purely based on that *raag* was taken. Then, a sample of 40 songs of unknown *raag* was taken and the transition probability matrices (TPM) using Markov chains were evaluated. The TPMs of these songs were then compared to the songs which were purely based on the *raagas* and the ‘distance’ using linear algebra technique was calculated. The ‘distances’ were compared which led to the best matching *raagas* out of the 8 mentioned above. 6 of the above songs were based on more than one *raag* and we could not consider them in our project. Apart from that, no non-conformity was observed.

Additionally, computer software that uses the TPM algorithm to give the best matching *raag* of a song was developed.

The results indicate that it is possible for a layman to identify the *raag* of a song using the TPM algorithm. Thus, it can be seen that Indian Classical Music which is traditionally considered to be a form of art has a scientific approach.

Motivation

A good project team need to know how to harness the initial excitement that comes that comes with starting a project and use it to maintain motivation – leading to success throughout the project’s lifecycle. Wheather a project team is highly motivated to attain a project’s goal can make the difference between a smooth project progress and one that is blogged down with complications.

“Music gives a soul to the universe, wings to the mind, flight to the imagination and life to everything.” — Plato

In turbulent times nowadays when violence and extremism have crept into the human lives in almost every single sphere of lives, fine arts like music can calm down and sober the situation to an extent. Music helps an individual to relax and reflect upon the various facets of characteristics related to self, which not only supports the person to understand the gray areas and improve upon the same but also aid in the process of overall mental and personality development because the brain cells find a path to express through music. Music has got a wonderful healing power owing to the melody that it carries along with it and this aid in the process of mending the morose mood of a person. The most vital aspect of music is the emotional enhancement that it provides for any human being who listens to it.

Pune has a great tradition of music. Late Pt. Bhimsen Joshi, Prabha Atre, Anand Bhate, Rahul Deshpande are some of the eminent music artists from Pune. Various music concerts (*maifil*) are organized in Pune; which includes ‘Vasantotsav’, ‘Sawai Gandharva’. We all team members throughout the academic years, attended various concerts in Pune. We have a special emotion towards music in our life. Ancient Indians were deeply impressed by the spiritual power of music, which leads to the origination of Indian classical music.

In Indian classical music, *ragas* constitute specific combinations of tonic intervals potentially capable of evoking distinct emotions. A *raaga* composition is typically presented in two modes, namely, *alaap* and *cheez*. So we have decided to study the notations of Raag (as Raag plays vital role in the field of music). At last, an algorithm is developed to predict the Raag of the given song.

Objectives

- ❖ To understand the concept of Markov chains
- ❖ To study basic terms of Markov Process
- ❖ To study the applications of Markov Process in various fields
- ❖ To study the basic terms of Python
- ❖ To Understand the concept of Simulation and Computing Order of Markov Chain using R
- ❖ To understand the concept of Second Order Markov Chain
- ❖ To compute distance between two songs using TPM technique (with and without using Python)
- ❖ To predict the Raaga of given song

Contents

1	Fundamentals of Music.....	1
1.1	Basic Terms.....	1
1.2	Concept of Raagas.....	2
1.3	More about Raagas.....	3
1.4	Writing the Notations.....	4
2	Markov Chain Theory.....	5
2.1	Some Statistical Terms.....	5
2.2	Introduction.....	5
2.3	Definition.....	6
2.4	Application of Markov Chains in Music.....	7
3	Hypothesis Testing based on Markov Chains using R.....	8
3.1	Test to Verify Markovian Property.....	8
3.2	Test to Estimate the True Order of the Markov Chain Sequence	9
3.3	Test to Verify Time Homogeneity	11
3.4	Maximum Likelihood Estimation of Transition Probabilities	13
4	Computations and Working.....	15
4.1	Algorithm.....	15
4.2	Computations Using R Package and MS-Excel.....	16
4.2.1	Creating a Transition Matrix.....	16
4.2.2	Calculating Distance between Two Songs.....	20
4.3	Python Programming.....	23
4.3.1	Introduction.....	23
4.3.2	Working.....	23
4.4	More Sophisticated Way.....	29
5	Results and Interpretation.....	31
6	Kruskal – Wallis Test.....	34
7	Simulation using R.....	35
	Limitations.....	37
	Looking Into The Future.....	38
	Applications To Other Fields.....	39
	Appendix I.....	40
	Appendix II.....	42
	Bibliography	47

1. Fundamentals of Music

1.1 Basic Terms

To analyse the pattern in music, we should first understand basics of music thoroughly. So, we are providing some terminologies of Indian classical music with suitable illustrations.

Indian classical music is a genre of South Asian music. It has two major traditions. The North Indian classical music tradition is called *Hindustani*, while the South Indian expression is called *Carnatic*. The roots of the classical music of India are found in the Vedic literature of Hinduism and the ancient *Natyashastra*, the classic Sanskrit text on performance arts by *Bharata Muni*.

Swara is a Sanskrit word that connotes a note in the successive steps of octave. A *swara* is the selected pitches from which the musicians construct scales, melodies and *raagas*.

Shruti is the smallest gradation of pitch that a human ear can detect and a singer or instrument can produce.

Note Name	Notation ID	Western Equivalent	Full Name
sa	Sa	Unison	Shadja
re (komal)	re	Minor second	Rishabha (komal)
re (shuddha)	Re	Major second	Rishabha (shuddha)
ga (komal)	ga	Minor third	Gandhaara (komal)
ga (shuddha)	Ga	Major third	Gandhaara (shuddha)
ma (shuddha)	Ma	Perfect fourth	Madhyama (shuddha)
ma (teevra)	MA	Augmented fourth	Madhyama (teevra)
pa	Pa	Perfect fifth	Panchama
dha (komal)	dha	Minor sixth	Dhaivata (komal)
dha (shuddha)	Dha	Major sixth	Dhaivata (shuddha)
ni (komal)	ni	Minor seventh	Nishaada (komal)
ni (shuddha)	Ni	Major seventh	Nishaada (shuddha)
Sa	Sa	Octave	Shadja

The seven notes of the musical scale in Indian classical music are shown in above table with their western equivalents. Collectively these notes are known as the *Sargam*. The *swara* concept is found in Chapter 28 of the ancient *Natya Shastra*.

तत्र स्वराः –
षड्जश्च ऋषभश्चैव गान्धारो मध्यमस्तथा ।
पञ्चमो धैवतश्चैव सप्तमोऽथ निषादवान् ॥ २१ ॥

— *Natya Shastra*, 28.21

These seven *swara* are shared by both major *raag* systems of Indian classical music, that is the *Hindustani* and *Carnatic*. In the general sense *swara* means tone, and applies to chanting and singing.

1.2 Concept of Raagas:

A *raag* is similar to a melodic mode in Indian classical music. While the *raag* is a remarkable and important feature of classical Indian music tradition, it has no direct translation to concepts in the classical European music tradition. Each *raag* is an array of melodic structures with musical image, considered in the tradition to have the ability to “colour the mind” and affect the emotions of the audience. A *raag* consists of at least five notes, and each *raag* provides the musician with a musical framework.

Aroha is the ascending scale of notes in a *raag*. The pitch increases as we go up from *Shadja* (Sa) to the *Taar Shadja* (Sa), possibly in a crooked manner.

Example - **Sa Re Ga Ma Pa Dha Ni Sa**

Avroha is the descending scale of notes in a *raag*. It is exactly opposite of *Aroha*.

Example - **Sa Ni Dha Pa Ma Ga Re Sa**

All *raagas* must include the root note Sa. All *raagas* must include at least one or both of the notes Ma (the fourth) or Pa (the fifth). There are more than 300 *raagas* in Indian classical music. Some of them are *Bageshree*, *Bhairavi*, *Bhup*, *Des*, *Kafi*, *Khamaj*, *Malhar*, *Malkaush*, *Pilu*, *Yaman*.

Bandish: According to Wikipedia, *Bandish*, *cheez* or *gatis* a fixed, melodic composition in Hindustani vocal or instrumental music. It is set in a specific *raag*, performed with rhythmic accompaniment by a *tabla* or *pakhavaj*, a steady drone, and melodic accompaniment by a *sarangi*, violin or harmonium. There are different ways of systematizing the parts of a composition. A *bandish* provides the literature element in the music, for standard structured singing. In the past many *gharanas* protected their *bandishes* from moving out of the family with gross incoherent vocal renditions. In the realm of vocal music, it is often known as *Cheez*.

Some terms related to Bandish are -

Asthayi: The initial phrase or line of a fixed, melodic composition.

Antara: The first body phrase or line of a fixed, melodic composition.

Sanchari: The third body phrase or line of a fixed, melodic composition, seen more typically in *Dhrupad bandish*.

Aabhog: The fourth and concluding body phrase or line of a fixed, melodic composition, seen more typically in *Dhrupad bandish*.

1.3 More about Raagas

Here, we are providing *Aroha* and *Avroha* of the *raagas* that are considered for the project. These *raagas* are selected on the basis of popularity and availability of the songs. All the following material is taken from authorised books of Gandharva Mahavidyalaya.

1) *Bageshree*

Aroha: ni Sa ga Ma Dha ni Sa

Avroha: Sa ni Dha Ma Pa Dha Ma ga Re Sa

2) *Bhimpalas*

Aroha: ni Sa ga Ma Pa ni Sa

Avroha: Sa ni Dha Pa Ma ga Re Sa

3) *Bhup or Bhupali*

Aroha: Sa Re Ga Pa Dha Sa

Avroha: Sa Dha Pa Ga Re Sa

4) *Des*

Aroha: Ni Sa Re Ma Pa Ni Sa

Avroha: Sa ni Dha Pa Dha Ma Ga Re Ga Ni Sa

5) *Durga*

Aroha: Sa Re Ma Pa Dha Sa

Avroha: Sa Dha Pa Ma Re dha Sa

6) *Kafi*

Aroha: Sa Re ga Ma Pa Dha ni Sa

Avroha: Sa ni Dha Pa Ma ga Re Sa

7) *Khamaj*

Aroha: Sa Ga Ma Pa Dha Ni Sa

Avroha: Sa Ni Dha Pa Ma Ga Re Sa

8) *Yaman (Kalyan)*

Aroha: Ni Re Ga Ma Dha Ni Sa

Avroha: Sa Ni Dha Pa Ma Ga Re Sa

1.4 Writing the Notations

Shuddha swar: Sa Re Ga Ma Pa Dha Ni.

They are also called *Shadja, Rishabh, Gandhar, Madhyam, Pancham, Dhaivat & Nishad*.

Komal Swar: Out of these seven swars, Re Ga Dha and Ni become “*Komal*” when their frequency is lower than their *Shuddha* form. Their place on the keyboard or harmonium is half note previous to their *Shuddha Swar*’s note. In this project, we denote *Komal Swars* as re ga dha and ni.

Teevra Swar: Among the seven basic swars, only one, that is, Ma becomes *Teevra* when its frequency is higher than its *Shuddha* form and its place next to its *Shuddha Swar*’s place on the keyboard. We denoted *Teevra Swaras* as MA.

Therefore the sequence of these 12 Swars including their *Shuddha, Komal* and *Teevra* forms is Sa re Re ga Ga Ma MA Pa dha Dha ni Ni Sa*.

Saptak is a group of seven prominent notes on a keyboard or harmonium. There are three main *Saptaks* on a harmonium. The lower *saptak* is called ***Mandra*** or ***Kharjaa*** *Saptak*, the middle *saptak* is called ***Madhya*** *Saptak* and the upper *saptak* is called ***Taar*** *Saptak*.

Mandra Saptak							Madhya Saptak							Taar Saptak						
re(k) ga(k)		MA(t) dha(k) ni(k)					re(k) ga(k)		MA(t) dha(k) ni(k)					re(k) ga(k)		MA(t) dha(k) ni(k)				
																				
Sa	Re	Ga	Ma	Pa	Dha	Ni	Sa	Re	Ga	Ma	Pa	Dha	Ni	Sa	Re	Ga	Ma	Pa	Dha	Ni
C	D	E	F	G	A	B	C	D	E	F	G	A	B	C	D	E	F	G	A	B

2. Markov Chain Theory

2.1 Some Statistical Terms:

- 1) **Deterministic Process:** A process in which the present state completely determines the future state.
Example - Calculating bank account balance after a month (By adding interest and deposits).
- 2) **State Space:** The mathematical space S of a stochastic process is called its state space. The state space is defined using elements that reflect the different values that the stochastic process can take.
- 3) **Stochastic Process:** It can be imagined as a description for something random, which has a notion of time. In other words, it is a collection of random variables defined on a common probability space.
Examples - Bernoulli process, Poisson process, Markov process, etc.
- 4) **Markov process:** Markov process is a stochastic process used in decision problems in which the probability of transition to any future state depends on the current state and not the manner in which the specific state is reached.
Examples – a first order differential equation is Markovian.

2.2 Introduction:

Consider a process that has a value in each time period. Let X_n be its value in time period n , and suppose we want to make a probability model for the sequence of successive values $X_0, X_1, X_2, \dots$. The simplest model would probably be to assume that the X_n are independent random variables, but often such an assumption is clearly unjustified. For instance, starting at some time, suppose that X_n represents the price of one share of some company, at the end of n additional trading days. Then it certainly seems unreasonable to suppose that the price at end of day $n+1$ is independent of the prices on days $n, n-1, n-2$ and so on down to day 0. However, it might be reasonable to suppose that the price of at the end of trading day $n+1$ depends on the previous end-of-day prices only through the price at the end of day n .

Thus, it might be logical to assume that the conditional distribution of X_{n+1} given all the past end-of-day prices $X_n, X_{n-1}, \dots, X_0$ depends on these past prices only through the price at the end of day n . Such an assumption defines a Markov chain, a type of stochastic process that is mainly used in later analysis.

2.3 Definition¹:

Let $\{X_n, n = 0, 1, 2, \dots\}$ be a stochastic process that takes on a finite or countable number of possible values. Unless otherwise mentioned, this set of possible values of the process will be denoted by the set non-negative integers $\{0, 1, 2, \dots\}$. If $X_n = i$, then the process is said to be in state i at time n .

We suppose that whenever the process is in state i , there is fixed probability p_{ij} that it will next be in state j . That is, we suppose that

$$P[X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_1 = i_1, X_0 = i_0] = P[X_{n+1} = j \mid X_n = i] = p_{ij} \dots (1)$$

For all states $i_0, i_1, \dots, i_{n-1}, i, j$ and all $n \geq 0$. Such a stochastic process is known as a Markov chain.

Equation (1) may be interpreted as stating that, **for a Markov chain, the conditional distribution of any future state X_{n+1} , given the past states $X_0, X_1, X_2, \dots, X_{n-1}$ and the present state X_n , is independent of the past states and depends only on the present state.**

The probability of moving from one state to another or remaining in the same state during a single period is called the transition probability.

$$p_{ij} = P[X_{n+1} = j \mid X_n = i]$$

The value p_{ij} represents the probability that the process will, when in state i , next make a transition into state j . Since probabilities are non-negative and since the process must make a transition into some state, we have

$$p_{ij} \geq 0, \quad i, j \geq 0; \quad \sum p_{ij} = 1, \quad i = 0, 1, \dots$$

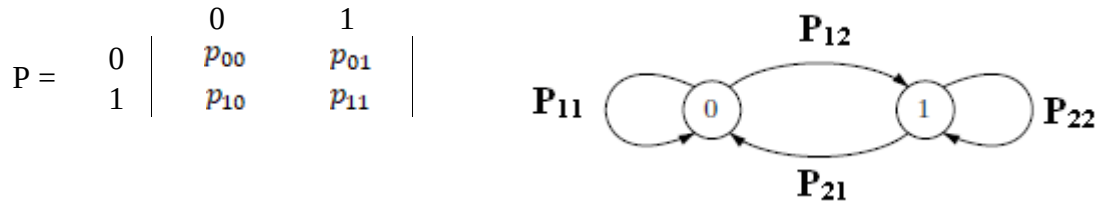
The transition probabilities can be arranged in a matrix of size $S \times S$ and such a matrix can be called as one step Transition Probability matrix represented as below:

$$P = \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1j} & \cdots & p_{1s} \\ p_{21} & p_{22} & \cdots & p_{2j} & \cdots & p_{2s} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ p_{i1} & p_{i2} & \cdots & p_{ij} & \cdots & p_{is} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ p_{s1} & p_{s2} & \cdots & p_{sj} & \cdots & p_{ss} \end{bmatrix} \quad \text{where, } S \text{ represents the no. of states.}$$

In general, any matrix $\mathbf{P}$, whose elements are non-negative and sum of the elements in either each row or column is unity, is called a Transition Matrix or Stochastic Matrix.

Illustration¹:

Consider a communication system that transmits the digits 0 and 1. Each digit transmitted must pass through several stages, at each of which there is probability p that the digit entered will be unchanged when it leaves. Let X_n denote the digit entering the n^{th} stage. Then $\{X_n, n = 0, 1, \dots\}$ is a two-state Markov chain having a transition probability matrix as



n-step transition probabilities:

It is defined as the probability that a process in state i will be in state j after n additional transitions. That is,

$$p_{ij}^{(n)} = P [X_{n+k} = j \mid X_k = i], \quad n \geq 0, \quad i, j \geq 0$$

The n -step transition matrix may be obtained by multiplying the matrix $\mathbf{P}$ by itself n times.

2.4 Application of Markov Chains in Music:

Markov Chains are used by many music software packages such as CSound, Max and SuperCollider for algorithmic music composition. In a first-order chain, the states of the system become note or pitch values, and a probability vector for each note is constructed, completing a transition probability matrix. A second-order Markov chain can be introduced by considering the current state and also the previous state, as indicated in the second table. Higher, n^{th} order chains tend to "group" particular notes together, while 'breaking off' into other patterns and sequences occasionally.

Further explanation about computations of probability transition matrices for an Indian song is given in the following chapters.

3. Hypothesis Testing Based on Markov Chains using R

In this chapter we will present three different tests, which we used to test the Markovian assumptions:

- A test to verify Markovian Property of given sequence of notations,
- A test to estimate the order of the Markov chain sequence.
- A test to verify if the transition probabilities are constant in time. (Time Homogeneity)

Below we present short description of each of these tests and give a summary of the results. To test the significance of the Markovian assumption, we use the chi-square test statistic. Note that test procedures and results are obtained using R commands.

3.1 Test to Verify Markovian Property

This test involves an analysis of three-state transition sequences. Three-state transition sequences consist of three-condition states: past, present and future. Possible transition sequences, with the same present and future states, but with different past states, are tracked to determine if there is a difference in occurrence dependent on the past state history. If there is no significant difference in frequency between the sequences being tracked, this may indicate the Markovian property.

If $X_1, X_2, \dots, X_N$ is a set of observations and n_{ijk} is the number of times t ($1 \leq t \leq N-2$) such that $X_t = i$; $X_{t+1} = j$; $X_{t+2} = k$, then if the Markov property holds n_{ijk} follows a binomial distribution with parameters n_{ij} and p_{jk} . A classical χ^2 test can check this distributional assumption.

We test the null hypothesis,

H₀: The sequence holds Markovian Property

vs.

H₁: The sequence does not hold Markovian Property.

The test statistic under H_0 is given by,

$$\chi^2 = \sum_i \sum_j \sum_k \frac{n_{ijk} - n_{ij} \hat{p}_{jk}}{n_{ij} \hat{p}_{jk}} \sim \chi^2_{(|S|^3)}$$

Where, $|S|$ is the cardinality of the state space.

Here we use R commands under the package “markovchain” to verify Markovian property of some sampled bandish.

```
> notes1.1<-c("ga", "ga", "Ma", "Dha", "Sa", "Sa", "Dha", "Ma", "ga", "Re",  
"Re", "ga", "Ma", "ga", "Re", "Sa", "Sa", "Dha", "Sa", "Sa", "Sa", "Re", "Sa",  
"Sa", "Sa", "Ma", "ga", "Re", "Sa", "Re", "Sa", "Sa", "Sa", "Ma", "Ma", "Ma",  
"Ma", "Ma", "Pa", "ga", "Ma", "Ma", "Dha", "Sa", "Sa", "Sa", "Sa", "Sa", "Sa",  
"Sa", "Dha", "Sa", "Sa", "Sa", "Ma", "ga", "Re", "Re", "Sa", "Sa", "Sa", "Sa",  
"Sa", "Sa", "Sa", "Sa", "Sa", "Sa", "Re", "Sa", "Sa", "Sa", "Dha", "Dha", "Sa",  
"Sa", "Sa", "Ma", "ga", "Re", "Sa", "Sa", "Sa", "Dha", "Ma", "ga", "Re", "Sa")
```

```
> verifyMarkovProperty(notes1.1)
Testing markovianity property on given data sequence
Chi - square statistic is: 21.83774 degrees of freedom are: 216 and
corresponding p-value is: 1
```

Similarly, we used the same procedure for all the bandish. Summary of results is given as follows:

Sr.no.	Bandish	Chi square value	df	p-value	Decision for H ₀
1	Bageshree	21.83774	216	1	Accept
2	Bhimpalas	34.44794	343	1	Accept
3	Bhup	20.89911	125	1	Accept
4	Des	34.059	512	1	Accept
5	Durga	32.20828	125	1	Accept
6	Kafi	46.98689	729	1	Accept
7	Khamaj	66.38904	512	1	Accept
8	Yaman	81.35202	512	1	Accept

Conclusion: In this case, as p-value is very large, we accept the hypothesis. This indicates that these sequences of notations follow Markovian Property. That is, all the notes are dependant on a previous note.

3.2 Test to Estimate the true Order of the Markov Chain Sequence

The first order Markov property can be judged by testing against the second order. Under the assumption of the second order, the ML estimate of $p_{ijk} = P[X_{t+2} = k | X_t = i, X_{t+1} = j]$ is given by $p_{ijk} = N_{ijk} / N_{ij+}$, where, $N_{ij+} = \sum N_{ijk}$ and

$$N_{ijk} = \sum_{t=1}^{T-2} I[X_t = i, X_{t+1} = j, X_{t+2} = k]$$

To test the hypothesis,

H₀: The sequence is of first-order,

vs.

H₁: The sequence is of second-order.

The corresponding χ^2 - statistic value for M states is given by,

$$\chi^2 = \sum_{ijk} \frac{(N_{ijk} - E_{ijk})^2}{E_{ijk}} \sim \chi^2_{(M-1)^2 M}$$

Where, $E_{ijk} = N_{i++} \hat{p}_{ij} \hat{p}_{jk}$

The above method of testing an order L against $L + 1$ is sequential in nature. We continue to test until an r -order model is not rejected against the $(r + 1)^{\text{th}}$ order model.

The test is very easily carried out using R command under the package “MOrder”.

```
> notes1.1<-c("ga", "ga", "Ma", "Dha", "Sa","Sa", "Dha", "Ma", "ga", "Re", "Re",
"ga", "Ma", "ga", "Re", "Sa", "Sa", "Dha", "Sa","Sa", "Sa", "Re", "Sa","Sa",
"Sa", "Ma", "ga", "Re", "Sa", "Re", "Sa","Sa", "Sa", "Ma", "Ma", "Ma", "Ma",
"Ma", "Pa", "ga", "Ma", "Ma", "Dha", "Sa","Sa", "Sa","Sa", "Sa", "Sa", "Sa",
"Dha", "Sa","Sa", "Sa", "Ma", "ga", "Re", "Re", "Sa", "Sa","Sa", "Sa",
"Sa","Sa", "Sa", "Sa","Sa", "Sa", "Re", "Sa", "Sa","Sa", "Dha", "Dha",
"Sa","Sa", "Sa", "Ma", "ga", "Re", "Sa", "Sa","Sa", "Dha", "Ma", "ga", "Re",
"Sa")
```

Notations of other songs are inputted in similar way

```
[1] "The order for the given sequence is: 1"
```

```
> getOrderPval(notes1.1)
      test1      test2 test3 test4 test5 test6
Order    "0"      "1"  "2"  "3"  "4"  "5"
P-values "3.40306188577442e-08" "1"  "1"  "1"  "1"  "1"
[1] "The given sequence is of First order"
```

```
> getOrderPval(notes2.1)
      test1      test2 test3 test4 test5 test6
Order    "0"      "1"  "2"  "3"  "4"  "5"
P-values "1.48924262522735e-19" "1"  "1"  "1"  "1"  "1"
[1] "The given sequence is of First order"
```

```
> getOrderPval(notes3.1)
      test1      test2      test3 test4 test5
Order    "0"      "1"      "2"  "3"  "4"
P-values "1.60852395635797e-07" "0.999999901248051" "1"  "1"  "1"
[1] "The given sequence is of First order"
```

```
> getOrderPval(notes4.1)
      test1      test2 test3 test4 test5 test6
Order    "0"      "1"  "2"  "3"  "4"  "5"
P-values "1.05630352368305e-10" "1"  "1"  "1"  "1"  "1"
[1] "The given sequence is of First order"
```

```
> getOrderPval(notes5.1)
      test1      test2      test3 test4 test5
Order    "0"      "1"      "2"  "3"  "4"
P-values "1.19600158601035e-05" "0.937730075562742" "1"  "1"  "1"
[1] "The given sequence is of First order"
```

```
> getOrderPval(notes6.1)
      test1      test2 test3 test4 test5 test6
Order    "0"      "1"  "2"  "3"  "4"  "5"
P-values "0.0187434000017706" "1"  "1"  "1"  "1"  "1"
[1] "The given sequence is of First order"
```

```
> getOrderPval(notes7.1)
      test1      test2 test3 test4 test5 test6
Order    "0"      "1"  "2"  "3"  "4"  "5"
P-values "1.19909408089909e-16" "1"  "1"  "1"  "1"  "1"
```

```
[1] "The given sequence is of First order"

> getOrderPval(notes8.1)
      test1      test2 test3 test4 test5 test6
Order    "0"      "1"   "2"   "3"   "4"   "5"
P-values "0.000153454673414188" "1"   "1"   "1"   "1"   "1"
[1] "The given sequence is of First order"

> getOrderPval(notes9.1)
      test1      test2 test3 test4 test5 test6
Order    "0"      "1"   "2"   "3"   "4"   "5"
P-values "1.76544879245721e-05" "1"   "1"   "1"   "1"   "1"
[1] "The given sequence is of First order"
```

Decision: For all the sequences, as p-value is very large, we accept H_0 under test for first order.

Conclusion: Thus, the Markov chain sequence is of First Order.

3.3 Test to Verify Time Homogeneity

The assumption of time homogeneity can be tested by dividing the data into k non-overlapping blocks of consecutive observations of moderate length. We then compute p_{ij} for each (i, j) obtained from each of such blocks and plot the estimators against the time. If the transition probabilities vary a lot over time, such plots would reveal changes.

In the stationary Markov chain (Time Homogeneous), p_{ij} is the probability that an individual was in state i at time $t-1$ and then moves to j at time t . A general alternative to this assumption is that the transition probability depends on t denoted by $p_{ij}(t)$.

We test the null hypothesis,
 $H_0: p_{ij}(t) = p_{ij}(t=1,2,3,\dots, T)$,
vs.
 $H_0: p_{ij}(t) \neq p_{ij}(t=1,2,3,\dots, T)$

$$\chi^2 = \sum_i \sum_{t,j} \frac{n_i(t-1)[\hat{p}_{ij}(t) - \hat{p}_{ij}]^2}{\hat{p}_{ij}}$$

follows chi-square distribution with $k(k-1)(T-1)$ degrees of freedom.

Where, k = no. of blocks, T = block size,

$$\hat{p}_{ij}(t) = \frac{n_{ij}(t)}{\sum_j n_{ij}}$$

$$n_{ij}(t-1) = \sum_j n_{ij}(t)$$

An R command to test this hypothesis is available under the package “markovchain”.

```
>notes2.1<-c("ni", "Dha", "Pa", "ga", "Ma", "Pa", "Pa", "Pa", "ga", "Sa", "ga",
"Ma", "ga", "Re", "Sa", "ni", "Sa", "ga", "Ma", "Ma", "Pa", "ga", "Ma", "Pa",
"ni", "Pa", "ni", "Sa", "ni", "Dha", "Pa", "ni", "Dha", "Pa", "ga", "Ma", "Pa",
"Pa", "Pa", "Pa", "Ma", "Pa", "ga", "Ma", "Pa", "ni", "ni", "Sa", "Sa", "Sa",
"Pa", "ni", "Sa", "ga", "Re", "Sa", "ni", "Sa", "ni", "Dha", "Pa", "ni", "Sa",
"ga", "Ma", "Pa", "Ma", "Pa", "ga", "Sa", "ga", "Ma", "Ma", "Ma", "ga", "Ma",
"Pa", "ni", "Dha", "Pa", "ga", "Ma", "Pa", "ni", "Sa", "ni", "Sa", "ga", "Ma",
"Pa", "ni", "Sa", "Sa", "Pa", "ni", "Sa", "Re", "Sa", "Sa", "Pa", "ni", "Sa",
"ni", "Sa", "Sa", "Pa", "ni", "Sa", "ga", "Re", "Sa")

>assessStationarity (notes2.1, 12)
The assessStationarity test statistic is: 115.2408 the Chi-Square d.f. are: 462
the p-value is: 1
```

Similarly, we tested for each bandish and obtained its p-value.

Sr.no.	Bandish	Chi square value	df	p-value	Decision for H ₀
1	Bageshree	79.61976	330	1	Accept
2	Bhimpalas	115.2408	462	1	Accept
3	Bhup	41.14422	120	1	Accept
4	Des	111.089	448	1	Accept
5	Durga	115.3041	300	1	Accept
6	Kafi	90.83038	504	1	Accept
7	Khamaj	180.5483	672	1	Accept
8	Yaman	86.72871	336	1	Accept

Conclusion: This result indicates that the sequences of notations of bandish we have considered are time homogeneous or stationary.

Thus, in this chapter we have verified Markovian assumptions required for further analysis of the data.

3.4 Maximum Likelihood Estimation of Transition Probabilities

In this paragraph we would like to introduce the Maximum Likelihood Method of Estimation. This method is used to estimate transition probabilities in a one-step transition probability matrix. A time homogeneous Markov chain can be fit from given data. An R command for ML Estimation is given as follows:

```
> mcFit<-markovchainFit(data=notes1.1)
> mcFit
$estimate
      Dha      Ma      Pa      Re      Sa      ga
Dha 0.1250000 0.2500000 0.0000000 0.0000000 0.6250000 0.0000000
Ma  0.1428571 0.3571428 0.0714285 0.0000000 0.0000000 0.4285714
Pa  0.0000000 0.0000000 0.0000000 0.0000000 0.0000000 1.0000000
Re  0.0000000 0.0000000 0.0000000 0.1818181 0.7272727 0.0909090
Sa  0.1162791 0.0930232 0.0000000 0.0697674 0.7209302 0.0000000
ga  0.0000000 0.3000000 0.0000000 0.6000000 0.0000000 0.1000000

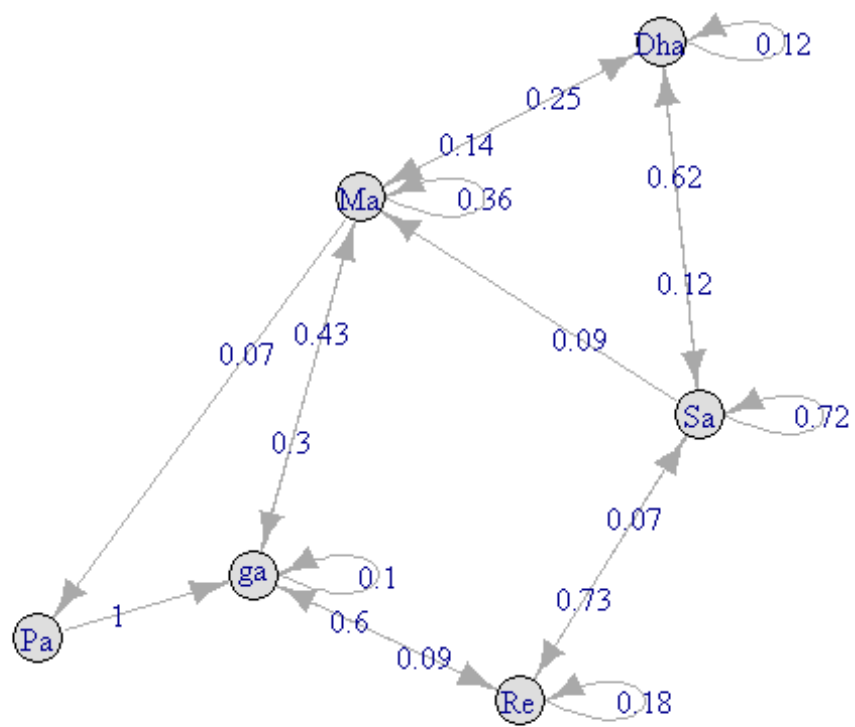
$standardError
      Dha      Ma      Pa      Re      Sa      ga
Dha 0.1250000 0.1767767 0.0000000 0.0000000 0.2795085 0.0000000
Ma  0.1010152 0.1597191 0.0714285 0.0000000 0.0000000 0.1749635
Pa  0.0000000 0.0000000 0.0000000 0.0000000 0.0000000 1.0000000
Re  0.0000000 0.0000000 0.0000000 0.1285648 0.2571297 0.0909090
Sa  0.0520015 0.0465116 0.0000000 0.0402802 0.1294829 0.0000000
ga  0.0000000 0.1732050 0.0000000 0.2449489 0.0000000 0.1000000

$confidenceLevel
[1] 0.95

$lowerEndpointMatrix
      Dha      Ma      Pa      Re      Sa      ga
Dha 0.0000000 0.0000000 0 0.0000000 0.1652494 0.000000
Ma  0.0000000 0.0944282 0 0.0000000 0.0000000 0.140782
Pa  0.0000000 0.0000000 0 0.0000000 0.0000000 0.000000
Re  0.0000000 0.0000000 0 0.0000000 0.3043319 0.000000
Sa  0.0307440 0.0165184 0 0.0035123 0.5079498 0.000000
ga  0.0000000 0.0151029 0 0.1970947 0.0000000 0.000000

$upperEndpointMatrix
      Dha      Ma      Pa      Re      Sa      ga
Dha 0.3306067 0.5407718 0.0000000 0.0000000 1.0000000 0.000000
Ma  0.3090125 0.6198575 0.1889181 0.0000000 0.0000000 0.7163609
Pa  0.0000000 0.0000000 0.0000000 0.0000000 0.0000000 1.000000
Re  0.0000000 0.0000000 0.0000000 0.3932886 1.0000000 0.2404412
Sa  0.2018141 0.1695281 0.0000000 0.1360226 0.9339106 0.000000
ga  0.0000000 0.5848970 0.0000000 1.0000000 0.0000000 0.2644854

$logLikelihood
[1] -79.68909
```



Sample Markov Chain plot

4. Computations and Working

For any statistical analysis, computations using the available data are a very important aspect, as any kind of error in it can ruin the objectives of the project.

Initially, computations had been carried out manually (For matrix calculations, online calculators are used), but as the data sets got bigger and bigger, doing all these manually became time consuming. So, to save the time and increase accuracy we have to use Python programming to write a code that will directly give us the computations by typing few commands. Some details about Python and the reasons behind choosing Python over other programming languages (C, C++, R) are discussed in the later half of this chapter. We divided this chapter in 2 sections -

- 1) Computations using R package and MS Excel
- 2) Computations using Python Programming

4.1 Algorithm²:

With credit given to Ala'a Wadi (Senior Honors Project, John Carroll Univ. 2012) and book "Linear Algebra with Applications" by Steven J. Leon, matrix norms were decided to be an adequate way to measure distance between two transition probability matrices. From which our algorithm is –

Given any two matrices A and B, the objective is to find the product $(B-A)^T(B-A)$, then find the trace of the resultant matrix and finally take the square root which will result in the matrix norm or the “distance” from A to B.

We want to examine what we know are two similar matrices and find the “distance” between them. The distance should be small and this algorithm turns out to accurately reflect our hypothesis. Application of this algorithm is provided in the next part of this chapter.

Ref.² Analysis of Music Note Patterns Via Markov Chains By Ala'a Wadi, Senior Honors Project (Fall 2012)

4.2 Computations using R Package and MS Excel

4.2.1 Creating a Transition Matrix

We have provided an illustration as follows.

Consider an example of the famous Hindi song from the movie ‘Paying Guest’

Song Name: Mana janaab ne pukaara nahi

Film: Paying Guest (1957)

Raag: Bhairavi

Composer: SD Burman

Singer: Kishore Kumar

Notations:

Lyrics:

Pa Pa Pa Pa dha Pa Ma Pa ni dha Pa

माना जनाब ने पुकारा नहीं

Pa Pa Pa Pa dha Pa Ma ga ga dha Pa Ma

क्या मेरा साथ भी गवारा नहीं

Ma Ma Ma Ma Pa Ma ga Ma Ma Ma Ma Pa Ma ga

मुफ्त में बन के, चल दिये तनके

Sa Sa Sa Sa Ma Ma Sa ga re Sa Sa re ga Ma Ma

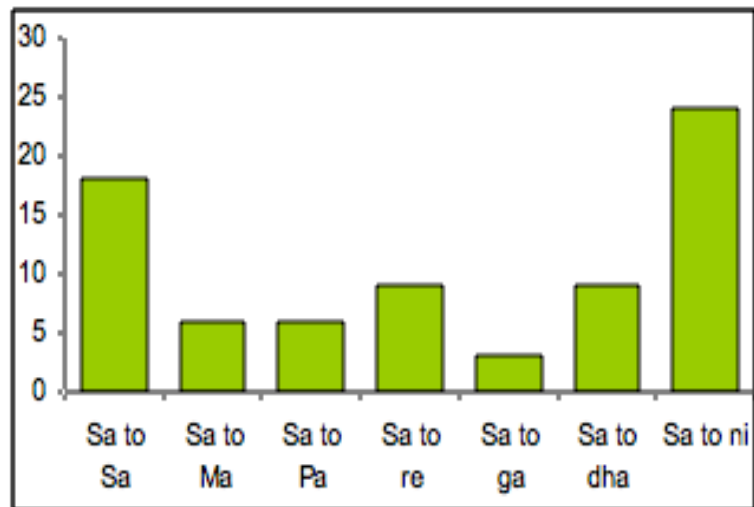
वल्ला जवाब तुम्हारा नहीं

Pa Pa Pa Pa dha Pa Ma Pa ni dha Pa...

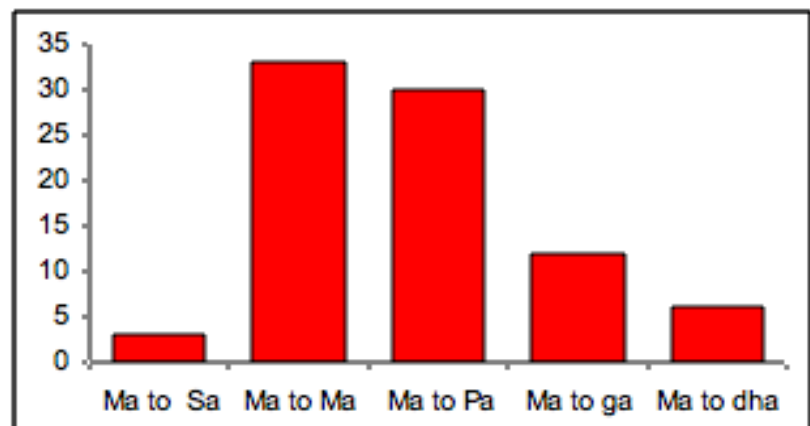
माना जनाब ने....

The following is the list of note transitions from the above song:

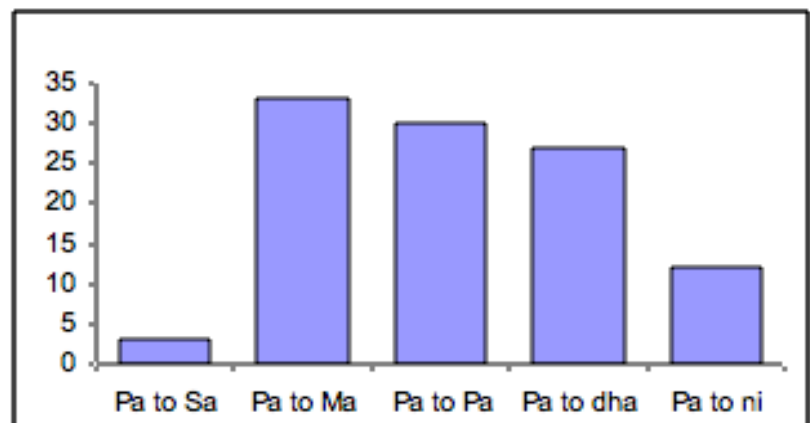
State Transition	Frequency
Sa to Sa	18
Sa to Ma	6
Sa to Pa	6
Sa to re	9
Sa to ga	3
Sa to dha	9
Sa to ni	24



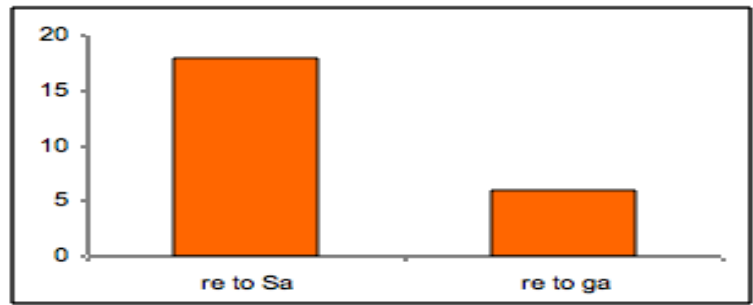
State Transition	Frequency
Ma to Sa	3
Ma to Ma	33
Ma to Pa	30
Ma to ga	12
Ma to dha	6



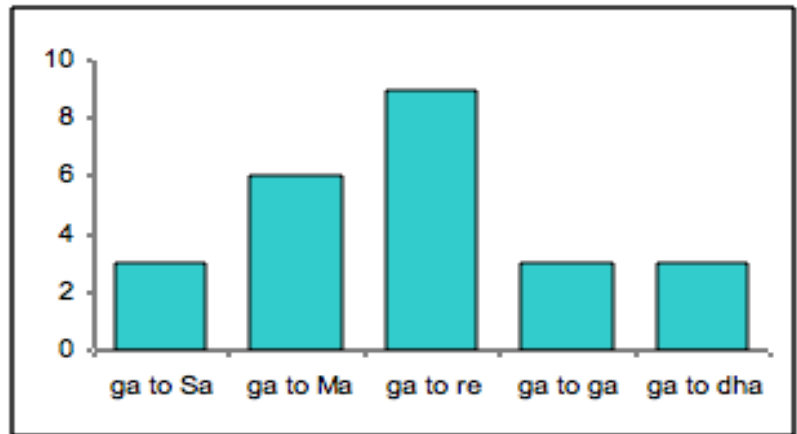
State Transition	Frequency
Pa to Sa	3
Pa to Ma	33
Pa to Pa	30
Pa to dha	27
Pa to ni	12



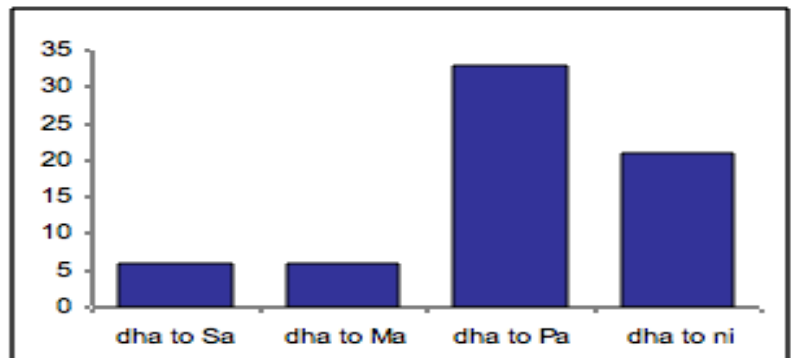
State Transition	Frequency
re to Sa	18
re to ga	6



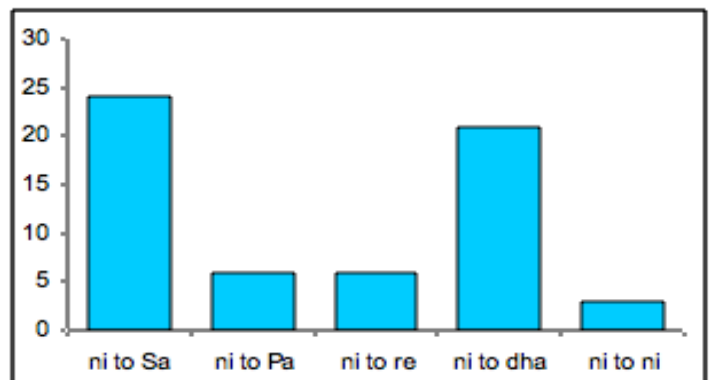
State Transition	Frequency
ga to Sa	3
ga to Ma	6
ga to re	9
ga to ga	3
ga to dha	3



State Transition	Frequency
dha to Sa	6
dha to Ma	6
dha to Pa	33
dha to ni	21



State Transition	Frequency
ni to Sa	24
ni to Pa	6
ni to re	6
ni to dha	21
ni to ni	3



The above frequency distribution tables are summarized into a 12x12 matrix with entries that denotes the frequencies of a specific note transitions to another note:

	Sa	Re	Ga	Ma	Pa	Dha	Ni	re	ga	MA	dha	ni
Sa	18	0	0	6	6	0	0	9	3	0	9	24
Re	0	0	0	0	0	0	0	0	0	0	0	0
Ga	0	0	0	0	0	0	0	0	0	0	0	0
Ma	3	0	0	33	30	0	0	0	12	0	6	0
Pa	3	0	0	33	30	0	0	0	0	0	27	12
Dha	0	0	0	0	0	0	0	0	0	0	0	0
Ni	0	0	0	0	0	0	0	0	0	0	0	0
re	18	0	0	0	0	0	0	0	6	0	0	0
ga	3	0	0	6	0	0	0	9	3	0	3	0
MA	3	0	0	0	0	0	0	0	0	0	0	0
dha	6	0	0	6	33	0	0	0	0	0	0	21
ni	24	0	0	0	6	0	0	6	0	0	21	3

Let **A** be a **transition matrix** obtained by dividing each value in the above matrix by the sum of its row's values. The entries in row of the matrix represent the probabilities of transition from one state (row note) to other state (column note). Every row of this final matrix contains values that add to 1.

	Sa	Re	Ga	Ma	Pa	Dha	Ni	re	ga	MA	dha	ni
Sa	0.24	0	0	0.08	0.08	0	0	0.12	0.04	0	0.12	0.32
Re	0	0	0	0	0	0	0	0	0	0	0	0
Ga	0	0	0	0	0	0	0	0	0	0	0	0
Ma	0.0357	0	0	0.3929	0.3571	0	0	0	0.1429	0	0.0714	0
Pa	0.0286	0	0	0.3143	0.2857	0	0	0	0	0	0.2571	0.1143
Dha	0	0	0	0	0	0	0	0	0	0	0	0
Ni	0	0	0	0	0	0	0	0	0	0	0	0
re	0.75	0	0	0	0	0	0	0	0.25	0	0	0
ga	0.125	0	0	0.25	0	0	0	0.375	0.125	0	0.125	0
MA	0	0	0	0	0	0	0	0	0	0	0	0
dha	0.0909	0	0	0.0909	0.5	0	0	0	0	0	0	0.3182
ni	0.4	0	0	0	0.1	0	0	0.1	0	0	0.35	0.05

4.2.2 Calculating distance between two songs:

In order to compare any two songs using transition matrices of both the songs we find the difference between these two matrices and then take the absolute value of the resulting “difference” matrix. This is done by taking square of the difference matrix. From this we have obtained ***Distance*** between those songs by taking square root of the trace.

The distance is the value that provides the biggest possible difference between the two matrices. Thus, if two songs have a small distance then the songs show resemblance. We provide an illustration below.

Note that all matrix operations and distance calculations were carried out using a computer program developed in **Python Language**. We will witness the details about it later.

Consider another song, we obtain transition matrix of the same.

Song Name: MastiBhara Hai Sama

Movie Name: Parvarish (1958)

Raag: Bhairavi

Composer: Dattaram

Singers: Manna Dey, Lata Mangeshkar

Let **B** be transition matrix of the second song obtained by dividing each value in the above matrix by the sum of its row's values.

B=		Sa	Re	Ga	Ma	Pa	Dha	Ni	re	ga	MA	dha	ni
	Sa	0.15	0	0	0.1	0	0	0	0.15	0	0	0.1	0.5
	Re	0	0	0	0	0	0	0	0	0	0	0	0
	Ga	0	0	0	1	0	0	0	0	0	0	0	0
	Ma	0.12	0	0.08	0.04	0.56	0	0	0	0.08	0	0.04	0.08
	Pa	0.04	0	0.09	0.61	0	0	0	0	0	0	0.13	0.13
	Dha	0	0	0	0	0	0	0	0	0	0	0	0
	Ni	0	0	0	0	0	0	0	0	0	0	0	0
	re	0.5	0	0	0.17	0	0	0	0	0.33	0	0	0
	ga	0.25	0	0	0	0	0	0	0.75	0	0	0	0
	MA	0	0	0	0	0	0	0	0	0	0	0	0
	dha	0.1	0	0	0.14	0.43	0	0	0	0	0	0.14	0.19
	ni	0.19	0	0	0.04	0	0.04	0	0	0	0	0.46	0.27

$$(A - B) = \begin{bmatrix} 0.0821 & 0 & 0 & -0.025 & 0.08 & 0 & 0 & -0.038 & 0.04 & 0 & 0.0674 & -0.206 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -0.089 & 0 & -0.083 & 0.3512 & -0.143 & 0 & 0 & 0 & 0.0596 & 0 & 0.0297 & -0.125 \\ -0.019 & 0 & -0.095 & -0.352 & 0.2857 & 0 & 0 & 0 & 0 & 0 & 0.1619 & 0.0191 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.25 & 0 & 0 & -0.167 & 0 & 0 & 0 & 0 & -0.083 & 0 & 0 & 0 \\ -0.125 & 0 & 0 & 0.25 & 0 & 0 & 0 & -0.375 & 0.125 & 0 & 0.125 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -0.076 & 0 & 0 & -0.02 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.096 \\ 0.2077 & 0 & 0 & 0 & 0.1 & 0 & 0 & 0.1 & 0 & 0 & -0.189 & -0.219 \end{bmatrix}$$

$$(A-B)^T = \begin{bmatrix} 0.082 & 0 & 0 & -0.089 & -0.019 & 0 & 0 & 0.25 & -0.125 & 0 & -0.076 & 0.208 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -0.083 & -0.095 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -0.025 & 0 & 0 & 0.351 & -0.352 & 0 & 0 & -0.167 & 0.25 & 0 & -0.02 & 0 \\ 0.08 & 0 & 0 & -0.143 & 0.286 & 0 & 0 & 0 & 0 & 0 & 0 & 0.1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -0.038 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -0.375 & 0 & 0 & 0.1 \\ 0.04 & 0 & 0 & 0.06 & 0 & 0 & 0 & -0.083 & 0.125 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.067 & 0 & 0 & 0.03 & 0.162 & 0 & 0 & 0 & 0.125 & 0 & 0 & -0.189 \\ -0.206 & 0 & 0 & -0.125 & 0.019 & 0 & 0 & 0 & 0 & 0 & 0.096 & -0.219 \end{bmatrix}$$

$$(A-B)^T(A-B) = \begin{bmatrix} 0.142 & 0 & 0.009 & -0.098 & 0.035 & 0 & 0 & 0.065 & -0.038 & 0 & -0.055 & -0.059 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.009 & 0 & 0.016 & 0.004 & -0.015 & 0 & 0 & 0 & -0.005 & 0 & -0.018 & 0.009 \\ -0.098 & 0 & 0.004 & 0.339 & -0.153 & 0 & 0 & -0.093 & 0.065 & 0 & -0.017 & -0.047 \\ 0.035 & 0 & -0.015 & -0.153 & 0.119 & 0 & 0 & 0.007 & -0.005 & 0 & 0.029 & -0.015 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.065 & 0 & 0 & -0.093 & 0.007 & 0 & 0 & 0.152 & -0.048 & 0 & -0.068 & -0.014 \\ -0.038 & 0 & -0.005 & 0.065 & -0.005 & 0 & 0 & -0.048 & 0.028 & 0 & 0.02 & -0.016 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -0.056 & 0 & -0.018 & -0.017 & 0.029 & 0 & 0 & -0.068 & 0.02 & 0 & 0.083 & 0.027 \\ -0.059 & 0 & 0.009 & -0.047 & -0.015 & 0 & 0 & -0.014 & -0.016 & 0 & 0.027 & 0.116 \end{bmatrix}$$

Trace of $(A-B)^T(A-B) = 2.9434$

Distance = 1.7156

Interpretation:

At this point, it was imperative that what value of distance would be considered large or small. Since both these songs belong to same *raag*, we may assume that these songs show resemblance.

Thus, in order to provide a better reference value of distance we have computed distance between pure *bandish* of different *raag*s.

4.3 Python Programming

4.3.1 Introduction

- **What is Python?**

Python is an interpreted, object-oriented, high level programming language with dynamic linguistics. Its high-level built in data structures, make it very attractive for Rapid Application Development. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Large organizations that use Python include Wikipedia, Google, Yahoo!, CERN, NASA and many more.

- **Why Python?**

Both Python and R are very useful. However, each of the language has its own set of applicability in Statistics and Data Science. For our project, we felt that Python is more suitable over R as program development is easier in Python than R. Following are some of the advantages of Python.

- i) Python is a fully featured programming language, which means it can be used to create real software.
- ii) R is a low level programming which can take longer codes than Python.
- iii) Python's syntax is more similar to the other languages and it reads more like human language.

4.3.2 Working with Python

Our Python program is developed by Atharva Vaidya, an engineering student from Sinhgad College of Engineering.

We will show the working of the program step by step using the screenshots.

Steps involved:

- (i) Accepting notations of a particular song from a user
- (ii) Obtaining Probability Transition Matrix of that song
- (iii) Accepting notations of a second song from a user
- (iv) Repeat step (ii) for second song
- (v) Calculating Trace and Distance between these two songs

The whole program code can be found out in the Appendix.

Now we will explain working using the screenshots. We will take same example that we have taken in the earlier section, so that we can easily verify the output with our manual calculations.

Note: All the working is carried out on Windows 10 HP PC using Python2 and Windows Powershell. There is no need of any programming prerequisites to understand and use the application as it is very simple and can be used by any person with decent computer skills.

Initializing the program and Accepting the notations of 1st song

```

PS C:\Python27> python base.py
Welcome!
Type "exit" to Enter Soor of Song B.
Enter the Soor of Song A:
Pa
Pa
Pa
Pa
dha
Pa
Ma
Pa
ni
dha
Pa
Pa
Pa
Pa
Pa
dha
Pa
Ma
ga
ga
dha
Pa
Ma
Ma
Ma
Ma
Ma
Pa
Ma
ga
Ma
Ma
Ma
Pa
Ma
ga
Sa
Sa
Sa
Sa
Ma
Ma
Sa
ga
re
Sa
Sa
re
ga
Ma
Ma
Pa
Pa
Pa
Pa
dha
Pa

```

```
Windows PowerShell
Pa
Ma
Pa
ni
dha
Pa
Sa
ni
Sa
dha
ni
Pa
dha
Ma
Pa
Ma
Pa
dha
ni
Sa
Pa
Ma
dha
Pa
ni
dha
Sa
ni
re
Sa
ni
Sa
dha
ni
Sa
ni
Sa
dha
ni
Pa
dha
Ma
Pa
Ma
Pa
dha
ni
Sa
Pa
Ma
dha
Pa
ni
dha
Sa
ni
re
Sa
Ma
Pa
dha
Pa
```

Note: There is no need to actually type all the notations in the Windows PowerShell. Notations can be copied directly from the text file or from the web. The conventions used for writing the notations is already explained in the 1st chapter.

Computing the TPM of 1st song

Windows PowerShell

```
Pa
dha
ni
Sa
Sa
re
ga
re
Sa
ni
ni
Sa
re
Sa
ni
dha
Sa
ni
dha
Pa
ni
dha
Pa
Ma
Ma
Ma
ga
re
Sa
Sa
exit
Matrix A
[[ 0.23999023 0. 0. 0.08001709 0.08001709 0. 0.
  0.11999512 0.04000854 0. 0.11999512 0.32006836]
 [ 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. ]
 [ 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. ]
 [ 0.03570557 0. 0. 0.39282227 0.35717773 0. 0.
  0. 0.14282227 0. 0.07141113 0. ]
 [ 0.027771 0. 0. 0.30566406 0.27783203 0. 0.
  0. 0. 0. 0.25 0.13891602]
 [ 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. ]
 [ 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. ]
 [ 0.75 0. 0. 0. 0. 0. 0.
  0. 0.25 0. 0. 0. ]
 [ 0.125 0. 0. 0.25 0. 0. 0.
  0.375 0.125 0. 0.125 0. ]
 [ 0. 0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. ]
 [ 0.13635254 0. 0. 0.09088135 0.5 0. 0.
  0. 0. 0. 0. 0.27270508]
 [ 0.39990234 0. 0. 0. 0.09997559 0. 0.
  0.09997559 0. 0. 0.35009766 0.04998779]]
Type "exit" to Display Trace and Distance.
```

Note: Due to the large order of Matrix (12x12), the arrangement of the elements of the matrix is not proper. We should accept this as one of the limitations.

Accepting notations of 2nd song

Windows PowerShell

```
0.09997559 0. 0. 0.35009766 0.04998779]]
Type "exit" to Display Trace and Distance.
Enter the Soor of Song B:
Sa
re
Sa
Ma
ga
Sa
Ma
Sa
re
Sa
dha
Pa
Ma
ga
re
ga
re
Ma
Pa
Ma
ni
ni
Ma
Pa
Ma
dha
dha
Ma
Pa
Ma
Sa
ni
dha
Sa
ni
dha
Pa
Ma
Pa
Ma
Sa
Sa
ni
Sa
ni
dha
Pa
Ma
Pa
ni
dha
Sa
ni
dha
ni
Sa
dha
dha
dha
```

Select Windows PowerShell

```
dha
ni
dha
Pa
Ma
Ga
Ma
Pa
Ma
Pa
Ga
Ma
Pa
Sa
Sa
Sa
ni
Sa
ni
dha
Pa
Ma
Pa
ni
Dha
Sa
ni
dha
ni
Sa
ni
ni
ni
dha
Pa
Ma
Ga
Ma
Pa
Ma
Pa
Ga
Ma
Pa
ni
ni
dha
Pa
dha
Ma
ni
ni
ni
dha
Pa
dha
Ma
Ma
Pa
dha
ni
```

Calculating the TPM for 2nd song and Finding the Distance

Windows PowerShell

```
ni
Sa
re
ga
re
Sa
ni
dha
Pa
Ma
Pa
Ma
exit
Matrix B
[[ 0.15002441 0.          0.          0.09997559 0.          0.          0.
  0.15002441 0.          0.          0.09997559 0.5         ]
 [ 0.          0.          0.          0.          0.          0.          0.
  0.          0.          0.          0.          0.          ]
 [ 0.          0.          0.          1.          0.          0.          0.
  0.          0.          0.          0.          0.          ]
 [ 0.11999512 0.          0.08001709 0.04000854 0.56005859 0.          0.
  0.          0.08001709 0.          0.04000854 0.08001709]
 [ 0.04348755 0.          0.0869751  0.60888672 0.          0.          0.
  0.          0.          0.          0.13049316 0.13049316]
 [ 1.          0.          0.          0.          0.          0.          0.
  0.          0.          0.          0.          0.          ]
 [ 0.          0.          0.          0.          0.          0.          0.
  0.          0.          0.          0.          0.          ]
 [ 0.5         0.          0.          0.16662598 0.          0.          0.
  0.          0.33325195 0.          0.          0.          ]
 [ 0.25        0.          0.          0.          0.          0.          0.
  0.75        0.          0.          0.          0.          ]
 [ 0.          0.          0.          0.          0.          0.          0.
  0.          0.          0.          0.          0.          ]
 [ 0.09521484 0.          0.          0.14282227 0.4284668  0.          0.
  0.          0.          0.          0.14282227 0.19042969]
 [ 0.19226074 0.          0.          0.03845215 0.          0.03845215
  0.          0.          0.          0.          0.46142578 0.26928711]]
The Trace is:2.9434
Distance is:1.71562215391
PS C:\Python27>
```

From earlier section, we can see that the calculations are almost equal. Hence, we can say that there is no error in the coding of the program.

4.4 More Sophisticated way

To predict the *raag* of the given song, we developed another Python program that will give us the best matched *raag* (for which the distance is minimum from given song). This will suffice our main objective and also simplify the working.

For this, we selected pure *bandishi* of certain *raagas*. We stored the notations of these *bandish* in the computer hard drive and imported those text files to our python code. The detailed code can be found out in the appendix. Notations of *bandish* are also given in the appendix.

List of *raagas* we considered are as follows:

- ❖ Bageshree
- ❖ Bhimpalas
- ❖ Bhup
- ❖ Des
- ❖ Durga
- ❖ Kafi
- ❖ Khamaj
- ❖ Yaman

Snapshot of the Output

```
Windows PowerShell
PS C:\Python27\Project> python base2.py
Welcome!
The Trace is:5.3242           Distance is:2.30742686775
The Trace is:5.6172           Distance is:2.37006065323
The Trace is:3.582            Distance is:1.89262549122
The Trace is:4.9922           Distance is:2.23432036646
The Trace is:3.8477           Distance is:1.96154435331
The Trace is:5.8594           Distance is:2.42061459138
The Trace is:4.4492           Distance is:2.10931712884
The Trace is:1.8691           Distance is:1.36716517839
The Trace is:2.9824           Distance is:1.72696898496
1.36716517839                Song No.:Yaman
1.72696898496                Song No.:Yaman K
1.89262549122                Song No.:Bhup
1.96154435331                Song No.:Durga
2.10931712884                Song No.:Khamaj
2.23432036646                Song No.:Des
2.30742686775                Song No.:Bageshree
2.37006065323                Song No.:Bhimpalas
2.42061459138                Song No.:Kafi
First Best Match Raga:Yaman
PS C:\Python27\Project>
```

Note: The song taken as an example is *bandish* of Yaman .We can see that the best matched *raag* is Yaman. However, we do not guarantee 100% accuracy for filmy songs as those songs are rarely based on a single *raag*.

We also developed a program for 2nd order Markov Chain, but as we can see from the output, results are not so accurate. Therefore, there is need for finding the exact order of the Markov chain, which we already did in earlier Section.

```
Windows PowerShell
PS C:\Python27\Project> python base3.py
Welcome!
Distance           Song No.
1.52197445774      9
1.55937186873      8
1.58791825199      3
1.61051234084      5
1.6741369045       7
1.80440000416      4
1.81895402636      2
1.93194768951      1
2.19552100878      6
First Best Match Raga:Yaman K
PS C:\Python27\Project>
```

5. Results and Interpretation

	Bageshree			Bhimpalas			Bhup			Des		
	1	2	3	1	2	3	1	2	3	1	2	3
Bageshree 1	0	1.607	0.911	1.809	1.934	2.106	2.074	1.89	1.832	2.313	2.092	1.988
Bageshree 2	1.607	0	1.591	1.847	1.903	2.013	2.153	2.035	2.056	2.536	2.392	2.312
Bageshree 3	0.911	1.591	0	1.766	1.884	2.07	1.963	1.866	1.893	2.486	2.257	2.22
Bhimpalas 1	1.809	1.847	1.766	0	0.546	1.185	1.975	1.908	2.025	2.073	2.235	2.026
Bhimpalas 2	1.934	1.903	1.884	0.546	0	1.133	2.004	1.942	2.058	2.17	2.409	2.059
Bhimpalas 3	2.106	2.013	2.07	1.185	1.133	0	1.96	1.973	2.074	1.98	2.359	1.96
Bhup 1	2.074	2.153	1.963	1.975	2.003	1.96	0	0.669	0.883	2.214	2.339	1.887
Bhup 2	1.89	2.035	1.866	1.908	1.942	1.973	0.669	0	0.583	2.122	2.193	1.744
Bhup 3	1.832	2.056	1.893	2.025	2.057	2.074	0.883	0.583	0	2.089	2.166	1.674
Des 1	2.313	2.536	2.486	2.073	2.169	1.98	2.214	2.122	2.089	0	1.425	1.465
Des 2	2.092	2.392	2.257	2.235	2.409	2.359	2.339	2.193	2.166	1.425	0	1.606
Des 3	1.988	2.312	2.22	2.026	2.058	1.96	1.887	1.744	1.674	1.465	1.606	0
Durga 1	1.591	1.943	1.713	1.712	1.794	1.993	1.335	1.131	1.142	1.966	1.929	1.476
Durga 2	1.611	1.955	1.667	1.736	1.840	2.036	1.322	1.153	1.217	2.054	1.915	1.587
Durga 3	1.685	1.98	1.738	1.669	1.762	1.973	1.315	1.174	1.224	1.974	1.956	1.546
Durga 4	1.675	1.995	1.752	1.841	1.931	2.118	1.336	1.156	1.189	2.089	1.977	1.642
Durga 5	1.59	2.047	1.709	2.067	2.161	2.31	1.497	1.347	1.286	2.192	2.035	1.734
Kafi 1	1.195	1.668	1.566	1.804	2.013	2.138	2.075	1.91	1.865	2.034	1.952	1.906
Kafi 2	1.4	1.813	1.621	1.679	1.910	2.056	2.017	1.831	1.831	1.782	1.816	1.894
Kafi 3	1.898	2.29	2.159	2.218	2.359	2.135	2.343	2.259	2.223	2.124	2.255	1.919
Khamaj 1	2.299	2.56	2.371	2.093	2.224	1.871	2.226	2.169	2.191	1.494	1.766	1.899
Khamaj 2	2.004	2.247	1.96	1.932	2.089	1.84	1.964	1.872	1.949	1.544	1.662	1.88
Khamaj 3	2.082	2.408	2.112	2.078	2.171	1.84	2.085	1.974	2.002	1.792	2.048	1.892
Khamaj 4	1.969	2.545	1.988	2.32	2.450	2.197	2.179	2.047	1.966	1.805	1.896	2.086
Yaman 1	2.266	2.307	2.331	2.333	2.370	2.284	1.893	1.702	1.519	2.147	2.234	1.926
Yaman 2	2.448	2.495	2.512	2.417	2.438	2.316	2.038	1.941	1.815	2.076	2.389	1.885
Yaman 3	2.29	2.382	2.308	2.161	2.185	2.021	1.656	1.556	1.513	2.277	2.422	1.836
Yaman 4	2.366	2.402	2.382	2.174	2.204	2.19	1.793	1.642	1.657	2.313	2.449	1.903

Durga 2					Kafi 1			Khamaj 1				Yaman			
1	2	3	4	5	1	2	3	1	2	3	4	1	2	3	4
1.591	1.611	1.685	1.6747	1.59	1.195	1.4	1.898	2.299	2.004	2.082	1.969	2.266	2.448	2.29	2.366
1.943	1.955	1.98	1.9951	2.047	1.668	1.813	2.29	2.56	2.247	2.408	2.545	2.307	2.495	2.382	2.402
1.713	1.667	1.738	1.7522	1.709	1.566	1.621	2.159	2.371	1.96	2.112	1.988	2.331	2.512	2.308	2.382
1.712	1.736	1.669	1.8414	2.067	1.804	1.679	2.218	2.093	1.932	2.078	2.32	2.333	2.417	2.161	2.174
1.794	1.841	1.762	1.9319	2.161	2.014	1.911	2.359	2.225	2.09	2.171	2.45	2.37	2.438	2.186	2.204
1.993	2.036	1.973	2.1176	2.31	2.138	2.056	2.135	1.871	1.84	1.84	2.197	2.284	2.316	2.021	2.19
1.335	1.322	1.315	1.3365	1.497	2.075	2.017	2.343	2.226	1.964	2.085	2.179	1.893	2.038	1.656	1.793
1.131	1.153	1.174	1.1563	1.347	1.91	1.831	2.259	2.169	1.872	1.974	2.047	1.702	1.941	1.556	1.642
1.142	1.217	1.224	1.1891	1.286	1.865	1.831	2.223	2.191	1.949	2.002	1.966	1.519	1.815	1.513	1.657
1.966	2.054	1.974	2.0888	2.192	2.034	1.782	2.124	1.494	1.544	1.792	1.805	2.147	2.076	2.277	2.313
1.929	1.915	1.956	1.9769	2.035	1.952	1.816	2.255	1.766	1.662	2.048	1.896	2.234	2.389	2.422	2.449
1.476	1.587	1.546	1.6423	1.734	1.906	1.894	1.919	1.899	1.88	1.892	2.086	1.926	1.885	1.836	1.903
0	0.375	0.415	0.4886	0.712	1.511	1.494	2.107	2.096	1.72	1.784	1.936	1.813	1.977	1.819	1.895
0.375	0	0.328	0.4379	0.792	1.54	1.544	2.139	2.14	1.725	1.802	1.943	1.918	2.073	1.881	1.972
0.415	0.328	0	0.5201	0.848	1.537	1.535	2.126	2.102	1.726	1.813	1.984	1.922	2.039	1.846	1.939
0.489	0.438	0.52	0	0.796	1.602	1.585	2.198	2.19	1.8	1.891	1.987	1.907	2.038	1.909	1.993
0.712	0.792	0.848	0.7964	0	1.652	1.718	2.243	2.271	1.898	1.946	2	1.962	2.148	2.002	2.126
1.511	1.54	1.537	1.602	1.652	0	0.887	1.571	2.151	1.856	2.039	2.001	2.283	2.399	2.29	2.344
1.494	1.544	1.535	1.5855	1.718	0.887	0	1.773	1.981	1.71	1.972	1.895	2.233	2.333	2.238	2.256
2.107	2.139	2.126	2.1982	2.243	1.571	1.773	0	2.547	2.324	2.429	2.368	2.421	2.5	2.173	2.48
2.096	2.14	2.102	2.1902	2.271	2.151	1.981	2.547	0	1.018	1.21	1.415	2.313	2.259	2.343	2.37
1.72	1.725	1.726	1.8001	1.898	1.856	1.71	2.324	1.018	0	0.931	1.151	2.109	2.047	2.204	2.268
1.784	1.802	1.813	1.8911	1.946	2.039	1.972	2.429	1.21	0.931	0	1.357	2.074	2.101	2.151	2.234
1.936	1.943	1.984	1.9868	2	2.001	1.895	2.368	1.415	1.151	1.357	0	2.171	2.172	2.268	2.406
1.813	1.918	1.922	1.907	1.962	2.283	2.233	2.421	2.313	2.109	2.074	2.171	0	1.367	1.727	1.506
1.977	2.073	2.039	2.0377	2.148	2.399	2.333	2.5	2.259	2.047	2.101	2.172	1.367	0	1.804	1.614
1.819	1.881	1.846	1.9086	2.002	2.29	2.238	2.173	2.343	2.204	2.151	2.268	1.727	1.804	0	1.508
1.895	1.972	1.939	1.9932	2.126	2.344	2.256	2.48	2.37	2.268	2.234	2.406	1.506	1.614	1.508	0

Note: Notations of these bandish are given in Appendix.

Result Table

Song No.	Song	Composer	Actual Raag	Shortest Distance	Predicted Raag
1	Chandan Sa Badan	Kalyanji Anandji	Yaman	1.062040341	Yaman
2	Mala Lagli Konachi Uchki	Ram Kadam	Durga	1.690968657	Durga
3	Ghat Doivar	Vasant Prabhu	Khamaj	1.414213562	Khamaj
4	Kaata Rute	Pt. Jitendra Abhisheki	Bhimpalas	1.560623873	Bhimpalas
5	Khoya Khoya Chand	S. D. Burman	Bhimpalas	1.877602360	Bhimpalas
6	Na Tum Hame Jano	S. D. Burman	Khamaj	2.113017688	Khamaj
7	Aage Bhi Jaane Na Tu	Ravi	Mixed	1.293765096	Bhup
8	Chaudhvi Ka Chand	Ravi	Pahadi	1.877602360	Bhimpalas

Interpretation: As we can see from the above table, *raagas* of almost all the songs were predicted correctly. As song no. 7 is not based on single *raag*, our program gave the closest match *raag*. For song no. 8 which is based on *raag* Pahadi, we could not identify its *raag* correctly (We did not include Pahadi in our list of *raagas*).

6. Kruskal Wallis Test

The normality assumption is unjustified for the above observations obtained. Hence the usual F test Analysis of Variance cannot be used in this case. A non-parametric alternative called Kruskal Wallis Test is used for Analysis of Variance.

Let X_{ij} : Distance of j^{th} song from i^{th} Raaga.

To test the hypothesis,

H_0 : Effect on distance due to different Raagas does not differ significantly.

vs.

H_1 : Effect on distance due to different Raagas differs significantly.

The test statistic under H_0 ,

$$H = \frac{1}{S^2} \left[\sum_{i=1}^a \frac{R_i^2}{n_i} - \frac{N(N+1)^2}{4} \right]$$

Where , n_i : No. of observations in the i^{th} treatment.

N : Total no. of observations

R_{ij} : Rank of the observation

R_i : Sum of the ranks of the i^{th} treatment

S^2 : Variance of the ranks

The above test can be carried out using R commands as follows:

```
>Xij<-scan("dist.txt")
Read 784 items
>treatments<-c(rep((1:28),28))
>kruskal.test(Xij, treatments)

Kruskal-Wallis rank sum test

data: Xij and treatments
Kruskal-Wallis chi-squared = 138.34, df = 27, p-value < 2.2e-16
```

Decision: Since, $\chi^2_{\text{cal}} > \chi^2_{27, 0.05}$ and $\chi^2_{\text{cal}} > \chi^2_{27, 0.01}$, hence we reject H_0

Conclusion: Thus, the effects of different Raagas on songs differ highly significantly.

7. Simulation using R

```
mar1<- new("markovchain", states=c("Dha","ga","Ma","Pa","Re","Sa"),
transitionMatrix = m1, name = "mar1")
sim1<-rmarkovchain(n=100, object=mar1, t0="Sa")
sim1
[1] "Sa" "Sa" "Dha" "Dha" "Ma" "ga" "Re" "Sa" "Sa" "Sa" "Sa" "Sa" "Dha"
[13] "Sa" "Sa" "Sa" "Re" "Sa" "Sa" "Dha" "Sa" "Sa" "Sa" "Sa" "Re" "Sa"
[25] "Ma" "Dha" "Sa" "Sa" "Sa" "Sa" "Re" "ga" "Re" "ga" "Ma" "ga"
[37] "Re" "Sa" "Re" "Sa" "Re" "Sa" "Dha" "Ma" "Ma" "Ma" "ga" "Ma"
[49] "Dha" "Sa" "Sa" "Sa" "Sa" "Sa" "Sa" "Sa" "Sa" "Sa" "Sa" "Sa"
[61] "Dha" "Sa" "Sa" "Ma" "Dha" "Ma" "ga" "Re" "Sa" "Sa" "Sa" "Sa" "Ma"
[73] "ga" "Ma" "ga" "Re" "Sa" "Sa" "Sa" "Sa" "Re" "Sa" "Dha" "Sa"
[85] "Dha" "Sa" "Sa" "Sa" "Re" "Sa" "Sa" "Dha" "Sa" "Sa" "Sa" "Ma"
[97] "Dha" "Sa" "Sa" "Sa"

sim2<-rmarkovchain(n=100, object=mar2, t0="Sa")
sim2
[1] "Sa" "ga" "Re" "Sa" "ni" "Dha" "Pa" "ga" "Ma" "Ma" "Pa" "ni"
[13] "Dha" "Pa" "ga" "Ma" "Ma" "Ma" "Pa" "ga" "Ma" "Ma" "Pa" "ga"
[25] "Ma" "Ma" "Pa" "ga" "Sa" "Sa" "Sa" "Sa" "Sa" "ga" "Ma" "Pa"
[37] "ni" "Sa" "Sa" "ga" "Ma" "Pa" "ni" "Sa" "Sa" "ni" "Sa" "Pa"
[49] "ni" "Sa" "ni" "Dha" "Pa" "ni" "Sa" "ni" "Sa" "ga" "Ma" "Pa"
[61] "ni" "ni" "Sa" "Sa" "ni" "Dha" "Pa" "ga" "Ma" "Pa" "ni" "Sa"
[73] "Pa" "ni" "Sa" "Sa" "ga" "Sa" "ga" "Sa" "Pa" "ni" "Sa" "ni"
[85] "Sa" "ni" "Sa" "Sa" "ni" "Dha" "Pa" "ga" "Ma" "Ma" "Pa" "Ma"
[97] "Pa" "ni" "Sa" "ga"

sim3<-rmarkovchain(n=100, object=mar3, t0="Sa")
sim3
[1] "Dha" "Pa" "Dha" "Re" "Sa" "Sa" "Dha" "Pa" "Sa" "Dha" "Pa" "Ga"
[13] "Re" "Sa" "Sa" "Dha" "Pa" "Ga" "Re" "Pa" "Ga" "Ga" "Re" "Sa"
[25] "Dha" "Pa" "Ga" "Ga" "Pa" "Ga" "Ga" "Pa" "Dha" "Re" "Sa" "Dha"
[37] "Sa" "Sa" "Re" "Sa" "Sa" "Dha" "Sa" "Dha" "Re" "Sa" "Sa" "Sa"
[49] "Pa" "Dha" "Re" "Sa" "Pa" "Dha" "Sa" "Sa" "Dha" "Pa" "Ga" "Re"
[61] "Pa" "Ga" "Ga" "Re" "Pa" "Ga" "Re" "Sa" "Dha" "Pa" "Ga" "Ga"
[73] "Pa" "Ga" "Pa" "Ga" "Ga" "Pa" "Ga" "Pa" "Sa" "Dha" "Pa" "Ga"
[85] "Re" "Pa" "Ga" "Re" "Sa" "Sa" "Dha" "Sa" "Dha" "Sa" "Dha" "Sa"
[97] "Sa" "Dha" "Sa" "Sa"

sim4<-rmarkovchain(n=100, object=mar4, t0="Sa")
sim4
[1] "Ni" "Ni" "Sa" "Sa" "Re" "Ga" "Re" "Ga" "Sa" "Ni" "Sa" "Re"
[13] "Ma" "Pa" "Ni" "Sa" "Ni" "Ni" "Sa" "Re" "Sa" "Re" "Ma" "Pa"
[25] "Re" "ni" "Dha" "Ma" "Pa" "Ni" "Sa" "ni" "Dha" "Ma" "Pa" "Pa"
[37] "Pa" "Re" "Ma" "Ma" "Ga" "Re" "Ni" "Sa" "Sa" "Sa" "Sa" "Re"
[49] "Ma" "Ma" "Pa" "Ni" "Sa" "Re" "Ma" "Pa" "Pa" "Pa" "Dha" "Ma" "Pa"
[61] "Dha" "Ma" "Ma" "Ma" "Ma" "Ma" "Pa" "Ni" "Ni" "Ni" "Sa" "Re"
[73] "Ga" "Sa" "Re" "ni" "Dha" "Pa" "Pa" "Ni" "Sa" "Re" "Ma" "Pa"
[85] "Ni" "Sa" "Sa" "Re" "Sa" "Ni" "Sa" "Ni" "Sa" "Re" "Sa" "Ni"
[97] "Sa" "Re" "Ga" "Re"

sim5<-rmarkovchain(n=100, object=mar5, t0="Sa")
sim5
[1] "Dha" "Dha" "Sa" "Re" "Dha" "Sa" "Re" "Sa" "Dha" "Sa" "Sa" "Dha"
[13] "Sa" "Dha" "Sa" "Re" "Sa" "Dha" "Ma" "Ma" "Pa" "Dha" "Dha" "Ma"
[25] "Re" "Pa" "Ma" "Pa" "Dha" "Dha" "Sa" "Re" "Pa" "Ma" "Pa" "Ma"
```

```

[37] "Pa" "Dha" "Dha" "Sa" "Dha" "Sa" "Re" "Sa" "Dha" "Dha" "Sa" "Sa"
[49] "Dha" "Dha" "Dha" "Sa" "Dha" "Dha" "Ma" "Re" "Pa" "Dha" "Dha" "Ma"
[61] "Pa" "Dha" "Sa" "Dha" "Ma" "Pa" "Dha" "Ma" "Re" "Pa" "Ma" "Pa"
[73] "Ma" "Pa" "Dha" "Sa" "Dha" "Ma" "Dha" "Sa" "Sa" "Dha" "Ma" "Re"
[85] "Pa" "Pa" "Dha" "Ma" "Dha" "Dha" "Sa" "Dha" "Ma" "Dha" "Ma" "Pa"
[97] "Ma" "Pa" "Dha" "Dha"

```

```
sim6<-rmarkovchain(n=100, object=mar6, t0="Sa")
```

```

sim6
[1] "Re" "Ma" "ga" "Re" "Ma" "ga" "Re" "Ga" "Re" "Ma" "Pa" "Pa"
[13] "Dha" "Dha" "Pa" "Pa" "Dha" "Pa" "Ma" "Ma" "Ma" "Ma" "Dha" "Sa"
[25] "ni" "Dha" "Dha" "Pa" "Pa" "Dha" "Ma" "Dha" "ni" "Sa" "Re" "Sa"
[37] "Re" "Sa" "Re" "Sa" "Sa" "Sa" "Sa" "ni" "Sa" "ni" "Dha" "Dha"
[49] "Dha" "Pa" "Pa" "Re" "Ma" "Dha" "Pa" "Pa" "Dha" "Pa" "Dha" "ni"
[61] "Dha" "Dha" "Pa" "Re" "Ga" "Re" "Ma" "Ma" "Ma" "Ma" "Pa" "Dha"
[73] "Pa" "Dha" "ni" "Dha" "Ma" "ga" "Re" "Sa" "Ni" "Dha" "Dha" "Pa"
[85] "Re" "Ma" "Pa" "Dha" "Dha" "Pa" "Dha" "Pa" "Pa" "Dha" "Sa" "Sa"
[97] "Sa" "Ni" "Dha" "Dha"

```

```
sim7<-rmarkovchain(n=100, object=mar7, t0="Sa")
```

```

sim7
[1] "Sa" "Ni" "Ni" "Sa" "ni" "Dha" "Ma" "Ga" "Ma" "Pa" "Pa" "Ga"
[13] "Ma" "Ga" "Ma" "Ga" "Ma" "Ga" "Ma" "Pa" "Pa" "Pa" "Dha" "ni"
[25] "Dha" "ni" "Dha" "Pa" "Dha" "ni" "Dha" "Ni" "Sa" "Ni" "Sa" "Ni"
[37] "Sa" "Ga" "Ma" "Pa" "Pa" "Pa" "Pa" "Ga" "Ma" "Ga" "Ma" "Ga"
[49] "Ma" "Pa" "Dha" "Pa" "Ga" "Ma" "Pa" "Dha" "Ni" "Re" "Ni" "Sa"
[61] "Ni" "Sa" "ni" "Pa" "Dha" "Sa" "ni" "Dha" "Pa" "Pa" "Pa" "Dha"
[73] "Ni" "Ni" "Sa" "Ni" "Sa" "ni" "Dha" "Ni" "Sa" "Ni" "Sa" "Dha"
[85] "ni" "Dha" "Pa" "Pa" "Dha" "Ni" "Re" "Sa" "Dha" "ni" "Dha" "Sa"
[97] "Ni" "Ni" "Sa" "ni"

```

```
sim8<-rmarkovchain(n=100, object=mar8, t0="Sa")
```

```

sim8
[1] "Ni" "Ni" "Re" "Re" "Ga" "Re" "Sa" "Ga" "Re" "Pa" "Re" "Ni"
[13] "Ni" "MA" "Dha" "Pa" "Re" "Ni" "Ni" "Ni" "Sa" "Sa" "Ni" "Dha"
[25] "Pa" "Re" "Ga" "Ni" "Dha" "Pa" "Ma" "Ga" "Re" "Sa" "Ga" "MA"
[37] "Dha" "MA" "Ni" "Ni" "Dha" "Pa" "Ga" "Re" "Pa" "Ma" "Ga" "Ni"
[49] "Ni" "Re" "Ni" "Re" "Ga" "Pa" "Re" "MA" "Dha" "Pa" "Re" "Ga"
[61] "Pa" "Ma" "Ga" "Pa" "Re" "Ni" "Re" "Ni" "Ni" "Ni" "Ni" "Re"
[73] "Ga" "MA" "Dha" "Pa" "Re" "Pa" "Re" "MA" "Dha" "Ni" "Dha" "Pa"
[85] "Pa" "Re" "Re" "Ga" "Pa" "Re" "Ga" "Re" "Sa" "Ni" "Re" "Ni"
[97] "Re" "Sa" "Ga" "Pa"

```

```
sim9<-rmarkovchain(n=100, object=mar9, t0="Sa")
```

```

sim9
[1] "Dha" "Pa" "Ga" "Pa" "MA" "Ga" "Pa" "Ga" "Pa" "MA" "Ga" "Pa"
[13] "Dha" "Ni" "Dha" "Ni" "Re" "Sa" "Ni" "Dha" "Pa" "Pa" "Ga" "Re"
[25] "Sa" "Ni" "Re" "Ga" "Pa" "Pa" "Ga" "Pa" "Pa" "MA" "Ga" "Pa"
[37] "Sa" "Dha" "Ni" "Re" "Ga" "Pa" "Sa" "Dha" "Pa" "Pa" "Re" "Ga"
[49] "Re" "Ga" "Pa" "Ga" "Pa" "Ga" "Re" "Sa" "Sa" "Pa" "MA" "Pa"
[61] "Ga" "Re" "Ga" "Ga" "Pa" "Pa" "Sa" "Sa" "Pa" "Pa" "Pa" "Ga"
[73] "Pa" "MA" "Pa" "Sa" "Ni" "Dha" "Ni" "Dha" "Pa" "Ga" "Pa" "Pa"
[85] "Pa" "Pa" "MA" "Pa" "Pa" "Ga" "Pa" "Re" "Sa" "Sa" "Dha" "Pa"
[97] "Re" "Sa" "Sa" "Ni"

```

Limitations

- ❖ It is observed that, if we go to the higher order Markov chains, it will give more accurate results. However, due to the limitation of theoretical knowledge and unavailability of advanced computation tools, we restricted this project upto 2nd order.
- ❖ There are very few non-classical songs whose notations are easily available. Therefore, we have to contain ourselves with Classical or Semi-Classical songs. Very few Film songs are considered in the project.
- ❖ As lot of manual work is needed for typing the notations, there is a chance of error, which is inevitable.
- ❖ As the data is taken from various sources, the reliability of the analysis is largely depends upon the credibility of these sources.
- ❖ At this stage, we are not familiar with the advanced R programming. So, we are unable to perform some statistical tests of Markov Chains.

Looking Into the Future

This project gave us great learning experience, as it demanded lot of advanced tools, both theoretical and Computational. We were introduced with totally new things like Python Programming and R packages. Therefore, we are hoping to continue this work in the near future.

We successfully developed the computer tool for the determination of the raaga in this project. However, in this era of smart phones, we are willing to develop an Android app which will make working simpler. This will also help us to improve our algorithm as we will get more and more feedbacks from the society.

Currently, we are trying to collect more number of Film songs with their notations, as this will give us a great opportunity to extend this work to the Mainstream music Industry. Also, we are going to discuss the possibility of a development of software which will give the notations directly by reading audio file with some more IT experts.

In this project, we restricted ourselves with just 10 raagas. In the future, work can be extended for many more raagas which will be the real test of our tool.

We also want to do the work for higher order Markov chains which required more theoretical knowledge, perhaps after doing the course of Stochastic Process in the final semester of B.Sc, we can apply more rigorous theory.

Applications to Other Fields

Text Mining:

- i) To solve the authorship dispute for the literary writings, Drama, Novels etc.
- ii) To identify the truthfulness of claim regarding the originality of articles of renowned personalities.

Genomics:

To identify the homogeneity between two Genome sequences via similarity measure which can be applied and useful in Genotyping, Serotyping and Drug designing.

Finance:

To identify the homogeneity between pattern in two Stocks by converting data into the sequence of the states. Also to study relative behaviour similarity between two Stock exchanges and two economies.

Appendix I

Notations of Bandish

➤ Bageshree

*ga ga Ma Dha ni Dha Ma ga Re Re ga Ma ga Re Sa Sa Dha ni Sa Re ni Sa Ma ga Re Sa Re
ni Sa Ma Ma Ma Ma Ma Pa ga Ma Ma Dha ni ni Sa Sa Sa Dha ni Sa Ma ga Re Re Sa ni Sa
ni Sa ni Sa Re Sa ni Dha Dha ni Sa Ma ga Re Sa ni Dha Ma ga Re Sa*

➤ Bhimpalas

*ni Dha Pa ga Ma Pa Pa Pa ga Sa ga Ma ga Re Sa ni Sa ga Ma Ma Pa ga Ma Pa ni Pa ni Sa
ni Dha Pa ni Dha Pa ga Ma Pa Pa Pa Pa Ma Pa ga Ma Pa ni ni Sa Sa Sa Pa ni Sa ga Re
Sa ni Sa ni Dha Pa ni Sa ga Ma Pa Ma Pa ga Sa ga Ma Ma Ma ga Ma Pa ni Dha Pa ga Ma
Pa ni Sa ni Sa ga Ma Pa ni Sa Sa Pa ni Sa Re Sa Sa Pa ni Sa ni Sa Sa Pa ni Sa ga Re Sa*

➤ Bhup

*Sa Sa Dha Sa Dha Pa Ga Pa Ga Re Sa Sa Re Pa Ga Ga Pa Dha Sa Dha Sa Dha Pa Ga Re
Sa Pa Ga Ga Pa Sa Dha Sa Sa Sa Dha Re Sa Ga Re Sa Dha Pa Dha Re Sa Dha Sa Dha Sa
Dha Pa Ga Re Sa*

➤ Des

*Ma Ma Pa Ni Sa Re ni Dha Pa Dha Ma Ma Ga Re Ga Sa Ni Sa Re Ga Sa Re Ma Ma Pa Pa
Ni Sa Ni Sa Re Sa ni Dha Ma Ma Ma Pa Ni Sa Sa Re Ni Sa Sa Ni Ni Ni Sa Sa Ni Sa Ni Sa
Re Sa ni Dha Ma Pa Re Ma Re Ma Pa Dha Ma Pa Pa Ni Sa Re Sa Re ni Dha*

➤ **Durga**

*Ma Pa Dha Sa Dha Ma Dha Ma Re Pa Ma Pa Dha Sa Dha Ma Ma Re Dha Sa Sa Re Ma Pa
Dha Sa Re Sa Dha Ma Dha Dha Ma Re Pa Ma Pa Dha Sa Dha Ma Dha Ma Pa Dha Sa Sa
Dha Sa Re Sa Dha Ma Dha Dha Dha Dha Dha Sa Dha Dha Ma Re Pa Pa*

➤ **Kafi**

*Re Ma Pa Dha Pa Ma ga Re Ma Ma Pa Pa Pa Re Ma Dha Dha Dha ni Dha Pa Dha Ma Pa
Pa Dha ni Sa ni Dha Pa Dha Ma Pa Dha Sa Sa Re Ga Re Sa Ni Dha Sa Sa Sa Sa Re Sa ni
Dha Pa Pa Dha ni Sa ni Dha Pa Re Ma*

➤ **Khamaj**

*Sa ni Dha Pa Dha Pa Ma Pa Ga Ma Pa Pa Ga Ma Ga Sa Ga Ma Pa Dha Ni Sa ni Dha Pa
Dha Ga Ma Ga Sa Ni Sa Dha ni Dha ni Pa Dha ni Dha Ni Sa Ni Sa Re Sa ni Dha Ma Ga
Ma Pa Pa Pa Ni Dha Sa Sa Ni Re Sa Ni Ni Ni Sa Dha Ni Sa Re Ni Sa Dha Sa ni Dha Pa
Ma Ga Ma Pa Pa Pa Dha Sa ni Dha Ga Ma Ga Ni Sa Ni Sa Dha ni Dha ni Pa Dha ni Dha
Ni Sa Ni Sa Re Sa ni Dha*

➤ **Yaman**

*Ni Dha Pa Re Re MA Ga Pa Re Ga Re Ni Re Sa Ni Re Ga MA Ga Pa Pa Ma Ga Re Sa Ni
Re Sa Ga MA Dha Ni Dha Sa Sa Re Ni Dha MA Dha Pa Pa Ga MA Dha Ni Ni Ni Ni MA
Dha Ni Sa Ni Re Sa Ni Re Ni Pa Re Ga Re Re Ga Pa Re Ga Re Sa Ga MA Ni Dha Pa Re Re
Ni Re Pa Re Ga Ni Sa*

Appendix II

Python Program to Compute Distance between the two songs

```
import numpy as np
from math import sqrt

index = 0
matrixA = np.zeros((12, 12), 'Float16')
matrixB = np.zeros((12, 12), 'Float16')
matrixC = np.zeros((12, 12), 'Float16')
swarList = ['Sa', 'Re', 'Ga', 'Ma', 'Pa', 'Dha', 'Ni', 're', 'ga', 'MA', 'dha', 'ni']
baseSetA = {'Sa': [], 'Re': [], 'Ga': [], 'Ma': [], 'Pa': [], 'Dha': [], 'Ni': [], 're': [], 'ga': [], 'MA': [], 'dha': [], 'ni': []}
baseSetB = {'Sa': [], 'Re': [], 'Ga': [], 'Ma': [], 'Pa': [], 'Dha': [], 'Ni': [], 're': [], 'ga': [], 'MA': [], 'dha': [], 'ni': []}

def printDict(baseSet):
    for i in baseSet:
        if len(baseSet[i])>1:
            print i
            for j in baseSet[i][:-1]:
                print j

def probability(matrix):
    sum = 0
    for i in range(0, 12):
        sum = 0
        for j in range(0, 12): # To Find The Sum of the Row
            sum += matrix[i][j]
        for j in range(0, 12): # To Divide each Row with Sum
            if sum != 0:
                matrix[i][j] /= sum
    return matrix

def inputMatrix(matrix, baseSet):
    index = 0
    while 1:
        swar = raw_input()
        if index == 0:
            temp = swar
        else:
            if swar == 'exit':
                break
            matrix[swarList.index(temp)][swarList.index(swar)] += 1
            temp = swar

    # For Finding Difference between two Consecutive Swar
    for i in baseSet:
        if swar == i:
```

```

        if len(baseSet[i]) == 0:
            baseSet[i].append(index)
        else:
            x = baseSet[i].pop(-1)
            baseSet[i].append(index - x - 1)
            baseSet[i].append(index)
        index += 1
    #printDict(baseSet)
    return matrix

print "Welcome!"
print "Type \"exit\" to Enter Soor of Song B."
print "Enter the Soor of Song A:"

matrixA = probability(inputMatrix(matrixA, baseSetA))
print "Matrix A"
print matrixA

print "Type \"exit\" to Display Trace and Distance."
print "Enter the Soor of Song B:"

matrixB = probability(inputMatrix(matrixB, baseSetB))
print "Matrix B"
print matrixB

matrixC = np.subtract(matrixA, matrixB)
trace = np.trace(np.dot(np.transpose(matrixC), matrixC))
print "The Trace is:" + str(trace)
print "Distance is:" + str(sqrt(trace))

```

Program to predict Raag of the given song using first order Markov chain

```

import numpy as np
from math import sqrt

index = 0
LIMIT = 10
resultarray = []
toparray = []
matrixA = np.zeros((12, 12), 'Float16')
matrixB = np.zeros((12, 12), 'Float16')
matrixC = np.zeros((12, 12), 'Float16')
swarList = ['Sa', 'Re', 'Ga', 'Ma', 'Pa', 'Dha', 'Ni', 're', 'ga', 'MA', 'dha', 'ni']
baseSetA = {'Sa': [], 'Re': [], 'Ga': [], 'Ma': [], 'Pa': [], 'Dha': [], 'Ni': [], 're': [], 'ga': [], 'MA': [], 'dha': [], 'ni': []}
baseSetB = {'Sa': [], 'Re': [], 'Ga': [], 'Ma': [], 'Pa': [], 'Dha': [], 'Ni': [], 're': [], 'ga': [], 'MA': [], 'dha': [], 'ni': []}

```

```

FILEPATH = './samples/notes'
ragas = ['Bageshree', 'Bhimpalas', 'Bhup', 'Des', 'Durga', 'Kafi', 'Khamaj',
'Yaman', 'Yaman K']
resultragas = []

def printDict(baseSet):
    for i in baseSet:
        if len(baseSet[i]) > 1:
            print (i)
            for j in baseSet[i][:-1]:
                print (j)

def probability(matrix):
    sum = 0
    for i in range(0, 12):
        sum = 0
        for j in range(0, 12): # To Find The Sum of the Row
            sum += matrix[i][j]
        for j in range(0, 12): # To Divide each Row with Sum
            if sum != 0:
                matrix[i][j] /= sum
    return matrix

def inputMatrix(matrix, baseSet, ch):
    index = 0
    filename = FILEPATH + str(ch) + ".txt"
    file = open(filename, 'r')
    for swar in file.read().split():
        if index == 0:
            temp = swar
        else:
            matrix[swarList.index(temp)][swarList.index(swar)] += 1
            temp = swar

    # For Finding Difference between two Consecutive Swar
    for i in baseSet:
        if swar == i:
            if len(baseSet[i]) == 0:
                baseSet[i].append(index)
            else:
                x = baseSet[i].pop(-1)
                baseSet[i].append(index - x - 1)
                baseSet[i].append(index)

        index += 1
    #printDict(baseSet)
    return matrix

print ("Welcome!")
# choice = raw_input("Enter the number from 1 to " + str(LIMIT) + ":")
for i1 in range(1, LIMIT):
    matrixA = probability(inputMatrix(matrixA, baseSetA, i1))
    matrixB = probability(inputMatrix(matrixB, baseSetB, 10))
    matrixC = np.subtract(matrixA, matrixB)
    matrixA = np.zeros((12, 12), 'Float16')
    matrixB = np.zeros((12, 12), 'Float16')
    trace = np.trace(np.dot(np.transpose(matrixC), matrixC))
    matrixC = np.zeros((12, 12), 'Float16')

```

```

        print ("The Trace is:" + str(trace) + "\t\t\tDistance is:" +
str(sqrt(trace)))
        resultarray.append(sqrt(trace))
        toparray.append(sqrt(trace))

resultarray.sort()
for x in range(0, 9):
    print (str(resultarray[x]) + "\t\tSong No.:" +
str(ragas[toparray.index(resultarray[x])]))

print ("First Best Match Raga:" + str(ragas[toparray.index(resultarray[0])]))

```

Program to predict Raag of the given song using second order Markov chain

```

import numpy as np
from math import sqrt

index = 0
LIMIT = 10
resultarray = []
toparray = []
matrixA = np.zeros((12, 144), 'Float16')
matrixB = np.zeros((12, 144), 'Float16')
matrixC = np.zeros((12, 12), 'Float16')
matrixtemp = np.zeros((12, 144), 'Float16')
swarList = ['Sa', 'Re', 'Ga', 'Ma', 'Pa', 'Dha', 'Ni', 're', 'ga', 'MA', 'dha',
'ni']
FILEPATH = './samples/notes'
ragas = ['Bageshree', 'Bhimpalas', 'Bhup', 'Des', 'Durga', 'Kafi', 'Khamaj',
'Yaman', 'Yaman K']
resultragas = []

def printDict(baseSet):
    for i in baseSet:
        if len(baseSet[i]) > 1:
            print (i)
            for j in baseSet[i][:-1]:
                print (j)

def probability(matrix):
    sum = 0
    for i in range(0, 12):
        sum = 0
        for j in range(0, 144): # To Find The Sum of the Row
            sum += matrix[i][j]
        for j in range(0, 144): # To Divide each Row with Sum
            if sum != 0:
                matrix[i][j] /= sum
    return matrix

def inputMatrix(matrix, ch):
    index = 0
    filename = FILEPATH + str(ch) + ".txt"
    file = open(filename, 'r')

```

```

for swar in file.read().split():
    if index == 0:
        temp1 = swar
    elif index == 1:
        temp2 = swar
    else:
        temp3 = swar
        col = swarList.index(temp1) * 12 + swarList.index(temp2)
        matrix[swarList.index(temp3)][col] += 1
        temp1 = temp2
        temp2 = temp3
    index = index + 1
return matrix

print ("Welcome!")
# choice = raw_input("Enter the number from 1 to " + str(LIMIT) + ':')
for i1 in range(1, LIMIT+1):
    matrixA = probability(inputMatrix(matrixA, i1))
    matrixB = probability(inputMatrix(matrixB, 10))
    matrixC = np.subtract(matrixA, matrixB)
    matrixA = np.zeros((12, 144), 'Float16')
    matrixB = np.zeros((12, 144), 'Float16')
    trace = np.trace(np.dot(matrixC, np.transpose(matrixC)))
    matrixC = np.zeros((12, 12), 'Float16')
    # print ("The Trace is:" + str(trace) + "\t\t\t\tDistance is:" +
str(sqrt(trace)))
    resultarray.append(sqrt(trace))
    toparray.append(sqrt(trace))

resultarray.sort()
print ("Distance\t\tSong No.")
for x in range(1, 10):
    print (str(resultarray[x]) + "\t\t" + str(toparray.index(resultarray[x])+1))

print ("First Best Match Raga:" + str(ragas[toparray.index(resultarray[1])]))

```

R Function to Obtain nth Order Transition Matrix

```

#Function to obtain Nth order transition Matrix
>trans.matrix<-function(X,l=1)
{
    tt<-table(X,-c((ncol(X)-l+1):ncol(X)) , c(X,-c(1:l)))
    tt<-tt/rowSums(tt)
    return(tt)
}

>trans.matrix(as.matrix(notes1),1)

```

Bibilography

Books and Research articles:

- [1] Ala'a Wadi, "Analysis of Music Note Patterns Via Markov Chains," Senior Honours Project, John Carrol University, Ohio, USA, Fall 2012
- [2] Sheldon Ross, "Introduction to Probability Models," Academic Press, San Diego, 2014
- [3] Sheldon Ross, "Stochastic Processes," John Wiley & Sons, Asia, 2004
- [4] Steven J. Leon, "Linear Algebra with Applications," Pearson, 2009
- [5] Giorgio Alfredo Spedicato, Tae Seung Kang, Sai Bhargav Yalamanchi and Deepak Yadav, "The markovchain Package: A package for Easily Handling Discrete Markov Chains in R"
- [6] Monika Skuriat-Olechnowska, "Statistical Inference and Hypothesis Testing for Markov Chains with Interval Censoring," Delft University of Technology, The Netherlands, 2005
- [7] Douglas C. Montgomery, "Design and Analysis of Experiments," Wiley Student Edition, 2013
- [8] Paul Finkelstein, "Music Segmentation Using Markov Chain Methods," March 2011
- [9] Dima Volchenkov and Jean Rene' Dawin, "Musical Markov Chains," International Journal of Modern Physics: Conference Series, Vol. 16 (2012) 116-135
- [10] गांधर्वमहाविद्यालय, "प्रारंभिक, प्रवेशिकाप्रथमगायन- वादन," पुणे, २०१३
- [11] सुरेशगुजर, "मराठीगीतांचेनोटेशन," राजेशप्रकाशन, पुणे
- [12] चंद्रकांतसाने, "गातारहेमेरादिल (भाग२)," नितीनप्रकाशन, पुणे, २०१५
- [13] John Paul Mueller, "Beginning Programming with Python for Dummies," Wiley, 2015

Online Literature:

- [14] <http://www.bluebit.gr/matrix-calculator/>
- [15] <https://cran.r-project.org/web/packages/markovchain/markovchain.pdf>
- [16] <https://cran.r-project.org/web/packages/markovchain/vignettes/markovchainCrashIntro.pdf>
- [17] <https://stats.stackexchange.com/questions/147164/fit-and-evaluate-a-second-order-transition-matrix-markov-process-in-r>
- [18] <http://learnraagabasics.blogspot.in/>
- [19] <https://www.learnpython.org/>